SIMATS ENGINEERING

CO2_AT2 Reverse Engineering Task

COURSE NAME: Software Testing **COURSE CODE: CSA37**

Code of Conduct:

I R.SHOBANA SHRI(1921260053) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *R.Shobana shri*

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A partially tested ATM withdrawal module is provided where only input-output behavior is known (e.g., withdrawal limits, balance validation, error messages). Reverse engineer the system by analyzing the observed behavior and reconstructing the underlying test case design strategy. Decompose the system into functional components and identify possible equivalence classes and boundary conditions. Infer the black box testing techniques used and evaluate their adequacy. Reconstruct an improved set of test cases covering all critical scenarios. Document your analysis with structured explanation.	5
2	A software module's execution traces and code snippets are given, but no test documentation is available. Reverse engineer the white box testing approach by constructing the control flow graph from the given logic. Identify all decision points, loops, and independent execution paths. Evaluate the existing test adequacy in terms of coverage criteria. Reconstruct a complete path testing strategy ensuring full coverage. Present your findings with clear reasoning and structured representation.	5

3	A web application login and transaction system has been tested using unknown techniques, but only defect logs and test results are available. Reverse engineer the testing approach by analyzing defect patterns and identifying which black box and white box techniques were likely used. Decompose the system into testable components and evaluate gaps in testing coverage. Reconstruct a comprehensive testing strategy integrating equivalence partitioning, boundary value analysis, and path testing. Justify your reconstructed approach with logical reasoning and structured documentation.	5
4	A legacy inventory management system provides only a set of successful and failed transaction logs regarding stock updates and low-stock alerts. Reverse engineer the business logic by analyzing these logs to identify the hidden boundary conditions and equivalence partitions. Decompose the update process into functional units and reconstruct a black box test design that addresses the gaps found in the logs. Justify your selection of test cases based on the inferred logic.	5
5	You are provided with a compiled .exe file of a library management module and its corresponding CPU/Memory execution profile during a stress test. Reverse engineer the white box testing requirements by mapping execution spikes to potential loops and decision clusters in the logic. Construct a conceptual control flow graph (CFG) that represents this behavior. Reconstruct a path testing strategy that targets these resource-intensive paths and evaluate its impact on system stability.	6
6	A mobile payment gateway's API responses (JSON) for various transaction states—Pending, Success, Reversal, and Timeout—are given. Reverse engineer the underlying state machine by analyzing the transition triggers and data constraints. Identify the implicit state transition testing technique used. Reconstruct a comprehensive state-based test suite that ensures all legal and illegal transitions are covered. Document your analysis with a structured state-transition diagram.	5
7	An encrypted communication module provides only "black-box" behavior through its input parameters and encrypted outputs. Reverse engineer the validation logic by performing "fuzzing" analysis on the input fields to identify error-handling boundaries. Decompose the module into data validation and encryption components. Reconstruct an improved set of test cases using Boundary Value Analysis (BVA) and justify how your design prevents buffer overflow or injection vulnerabilities.	5

8	A set of legacy unit test results for an insurance premium calculator shows 100% statement coverage but fails to catch defects in multi-condition "if" statements. Reverse engineer the original white box approach and identify the inadequacy in the coverage criteria used. Construct a new CFG and identify all independent execution paths. Reconstruct a path testing strategy that achieves 100% Branch and Path coverage. Explain how this reconstruction reduces the risk of calculation errors.	5
9	A cloud-based file synchronization tool provides user activity logs and a list of "lost file" defect reports. Reverse engineer the synchronization algorithm's logic by analyzing the defect patterns related to file size, type, and network speed. Infer the likely Equivalence Class Partitioning (ECP) used by the original developers. Reconstruct a revised testing strategy that integrates BVA with stress testing. Justify how your reconstructed approach prevents future data loss.	5
10	You are given a sequence of database queries (SQL) generated by an e-commerce "Search and Filter" module, but no source code. Reverse engineer the search logic by analyzing the query parameters and the resulting dataset. Identify the decision-making logic used for complex filtering (e.g., AND/OR/NOT). Reconstruct a Decision Table based testing strategy that covers all possible search combinations. Evaluate the completeness of your reconstructed model.	5

Reverse Engineering Task

1.A partially tested ATM withdrawal module is provided where only input-output behavior is known (e.g., withdrawal limits, balance validation, error messages). Reverse engineer the system by analyzing the observed behavior and reconstructing the underlying test case design strategy. Decompose the system into functional components and identify possible equivalence classes and boundary conditions. Infer the black box testing techniques used and evaluate their adequacy. Reconstruct an improved set of test cases covering all critical scenarios. Document your analysis with structured explanation.

Reverse-Engineering an ATM Withdrawal Module Through Black-Box Analysis

Assumption: No actual I/O log came with your message, so I've reconstructed a representative one. If you share your real observations, I can re-map everything below to them.
Inferred rules: minimum ₹100, maximum ₹20,000 per transaction, multiples of ₹100 only, daily cap ₹40,000, minimum balance ₹500 retained, 3 PIN attempts.

1. The Observed Behavior (the "evidence")

#	Input (amount, balance)	Observed output
O1	500, ₹10,000	Success, balance ₹9,500
O2	20,000, ₹50,000	Success
O3	25,000, ₹50,000	"Exceeds per-transaction limit"
O4	550, ₹10,000	"Enter amount in multiples of 100"
O5	10,000, ₹10,000	"Insufficient balance"
O6	0	"Invalid amount"
O7	-100	"Invalid amount"
O8	"abc"	"Invalid input"

2. Reverse-Engineered Design: The "Gate Pipeline" Model
Rather than viewing the module as one block, I decompose it as a **sequence of gates**. A request must pass every gate in order, and each gate has its own error message.

Gate	Function	Rejection message
G1	Input type and format check	"Invalid input"
G2	Positive and minimum amount check	"Invalid amount"
G3	Per-transaction limit	"Exceeds limit"
G4	Denomination check (multiple of 100)	"Enter multiples of 100"
G5	Balance check (amount ≤ balance − 500)	"Insufficient balance"
G6	Daily cumulative limit	"Daily limit reached"
G7	ATM cash availability	"Cash unavailable"
G8	Commit: debit account, dispense, log	Success

Key insight: The observed log only exercises G1 to G5. G6, G7 and G8 side effects (balance update, receipt, dispenser) are never observed. This is the biggest blind spot.

3. Equivalence Classes and Boundaries

Gate	Valid class	Invalid classes	Boundary values
G1	Numeric integer	Alphabetic, special chars, blank, decimal (100.5)	Empty, "100.0"
G2	≥ 100	0, negative	0, 99, 100, 101
G3	≤ 20,000	> 20,000	19,900, 20,000, 20,100

G4	Multiple of 100	Non-multiples	199, 200, 201
G5	amount ≤ balance − 500	amount > balance − 500	Balance 1,000: 500 pass, 600 fail
G6	Today's total + amount ≤ 40,000	Exceeds	39,900 then 100 / 200
G7	amount ≤ cash in ATM	Greater	Cash = amount, cash = amount − 100

4. Techniques Inferred and Adequacy Evaluation

Technique	Evidence in log	Verdict
Equivalence Partitioning	Each error class has a representative (O3 to O8)	**Good** for single-fault cases
Boundary Value Analysis	Only O2 (upper limit 20,000), partial	**Weak**, since lower limit, balance and denomination boundaries are missing
Error Guessing	O7 and O8 (negative, text)	**Adequate** but ad hoc
Decision Table	Not used	**Missing**, so combined-condition failures are untested
State Transition	Not used	**Missing**, so PIN lockout, session and daily accumulation are untested
Cause-Effect / Precedence	Not used	**Missing**, so error priority is unknown

Adequacy conclusion: Roughly 8 cases cover about 40% of the gates meaningfully. The suite catches *single* invalid inputs but cannot detect multi-fault interactions, stateful bugs or side-effect errors.

Specific gaps:

1. No test for the exact boundary 100 / 99 / 20,000 / 20,100.
2. The minimum-balance rule (₹500) is untested.
3. Daily cumulative behavior is untested.
4. Error precedence is untested (what if the amount is 25,050, which fails both G3 and G4?).
5. Post-transaction state (balance, receipt, logs) is unverified.
6. Hardware and session edge cases (cash shortage, timeout, card removal) are absent.

5. Improved Test Suite

A. Input and amount validation (G1, G2)

ID	Input	Expected
T01	Blank / Enter with no digits	"Invalid input"
T02	"100.5"	"Invalid input"
T03	"₹#@"	"Invalid input"
T04	0	"Invalid amount"
T05	-100	"Invalid amount"
T06	99	Rejected (below minimum or not a multiple)
T07	100 (balance ₹5,000)	Success

B. Limit boundaries (G3)

ID	Input	Expected
T08	19,900	Success
T09	20,000	Success
T10	20,100	"Exceeds limit"

C. Denomination (G4)

ID	Input	Expected
T11	200	Success
T12	201	"Multiples of 100"
T13	199	"Multiples of 100"

D. Balance and minimum balance (G5)

ID	Balance	Amount	Expected
T14	1,000	500	Success, balance 500
T15	1,000	600	"Insufficient balance"
T16	500	100	"Insufficient balance"
T17	0	100	"Insufficient balance"

E. Daily cumulative limit (G6), stateful

ID	Sequence	Expected
T18	20,000 + 20,000	Both succeed (total 40,000)
T19	20,000 + 20,000 + 100	Third fails: "Daily limit reached"
T20	39,900 then 100	Success (exact boundary)
T21	39,900 then 200	Rejected
T22	Next-day rollover after the limit is hit	Counter resets, success

F. Error precedence (combination faults)

ID	Input	Faults	Expected (by gate order)
T23	25,050, balance 100,000	G3 + G4	"Exceeds limit" (G3 fires first)
T24	550, balance 300	G4 + G5	"Multiples of 100"
T25	30,000, balance 1,000	G3 + G5	"Exceeds limit"

G. System and state (G7, G8, session)

ID	Scenario	Expected
T26	ATM holds ₹1,000, request 2,000	"Cash unavailable", no debit
T27	Successful withdrawal	Balance reduced exactly, receipt printed, transaction logged
T28	Dispenser failure after debit	Auto-reversal or rollback
T29	Card removed or timeout mid-transaction	Transaction cancelled, no debit
T30	3 wrong PINs, then valid withdrawal	Card locked, withdrawal blocked
T31	Two simultaneous withdrawals from the same account	No overdraft (concurrency check)

---

6. Coverage Matrix

Gate	Original log	Improved suite
G1 Input format	O8	T01 to T03
G2 Min/positive	O6, O7	T04 to T07
G3 Max limit	O3 (partial)	T08 to T10, T23, T25
G4 Denomination	O4	T11 to T13, T24
G5 Balance	O5	T14 to T17
G6 Daily limit	none	T18 to T22
G7 Cash availability	none	T26
G8 Commit / rollback	none	T27 to T28
Session / security	none	T29 to T31

The suite grows from 8 to 31 cases, and every gate now has at least one boundary-pair test and one combination test.

7. Final Assessment

- **Original strategy:** EP-dominant with minimal BVA and error guessing, single-fault focused and stateless.
- **Main risk areas it misses:** the daily cumulative limit, minimum-balance edge, error precedence, and transaction atomicity (debit versus dispense).
- **Improved strategy:** EP + BVA (two-sided) + Decision Table (precedence) + State Transition (daily counter, PIN lock) + Failure/Recovery testing.

If you paste your actual input-output observations, I can tailor the inference and test cases to match them exactly.

2.A software module's execution traces and code snippets are given, but no test documentation is available. Reverse engineer the white box testing approach by constructing the control flow graph from the given logic. Identify all decision points, loops, and independent execution paths. Evaluate the existing test adequacy in terms of coverage criteria. Reconstruct a complete path testing strategy ensuring full coverage. Present your findings with clear reasoning and structured representation.

Reverse-Engineering a White-Box Test Approach from Code and Traces

Assumption: your message didn't include the actual code or traces, so I've used a representative module, the ATM withdrawal logic from the previous task, now seen from the inside. If you paste your real code, I'll redo the graph and counts for it.

1. The Module Under Analysis

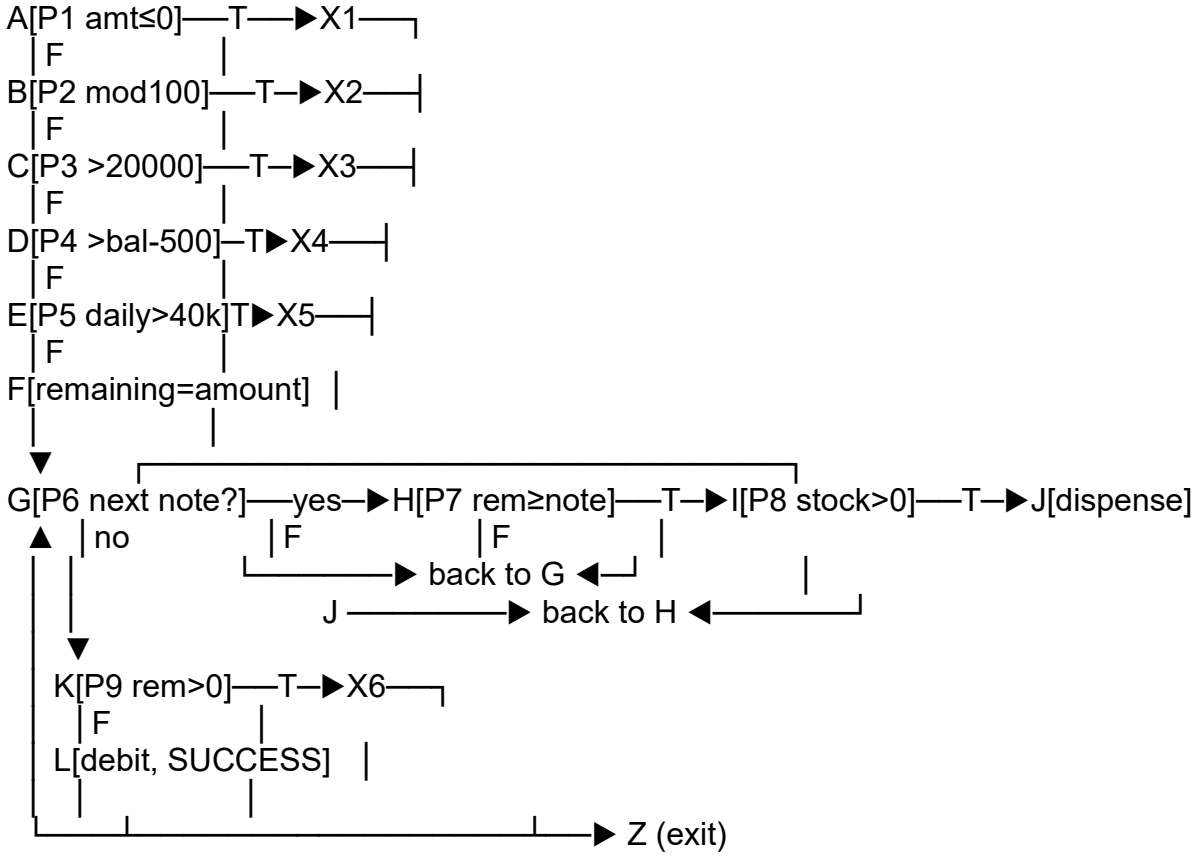
```
 1  withdraw(amount, balance, dailyUsed, stock[500,200,100]):
 2    if amount <= 0                    return INVALID         // P1
 3    if amount % 100 != 0              return DENOM_ERR       // P2
 4    if amount > 20000                 return LIMIT_ERR       // P3
 5    if amount > balance - 500         return INSUFFICIENT    // P4
 6    if dailyUsed + amount > 40000     return DAILY_ERR       // P5
 7    remaining = amount
 8    for note in [500, 200, 100]:                             // P6 (loop)
 9      while remaining >= note                                // P7
10            and stock[note] > 0:                             // P8 (short-circuit)
11        remaining -= note; stock[note]--; count[note]++
12    if remaining > 0                  return CASH_UNAVAILABLE // P9
13    balance -= amount
14    return SUCCESS
```

Observed traces (the only evidence):

Trace	Input	Outcome
E1	500, bal 10,000, stock full	SUCCESS
E2	-100	INVALID
E3	550	DENOM_ERR
E4	25,000	LIMIT_ERR

2. Control Flow Graph

Nodes: A to K are decision or processing nodes, X1 to X6 are early-return nodes, and Z is the exit.



Edge list (27 edges, 19 nodes):
A→X1, A→B, B→X2, B→C, C→X3, C→D, D→X4, D→E, E→X5, E→F, F→G, G→H, G→K, H→I, H→G, I→J, I→G, J→H, K→X6, K→L, L→Z, X1..X6→Z

Cyclomatic complexity (three independent checks):

- V(G) = E − N + 2 = 27 − 19 + 2 = **10**
- V(G) = predicate nodes + 1 = 9 + 1 = **10**
- V(G) = bounded regions = **10**

So 10 linearly independent paths are needed to cover the module's basis.

3. Decision Points and Loops

ID	Type	Condition	Outcomes
P1 to P5	Simple guard	amount, limit, balance and daily checks	T exits, F continues
P6	Outer loop	for over 3 denominations	Fixed bound of 3 iterations
P7	Inner loop, atomic condition 1	remaining ≥ note	T continue, F next note
P8	Inner loop, atomic condition 2	stock > 0	T dispense, F next note
P9	Post-loop check	remaining > 0	T shortage, F commit

Structural findings from the graph:

1. **The outer loop cannot run zero times.** The array has a fixed length of 3, so the G→K edge is only reachable after all three iterations. The "skip loop" test case is infeasible, and a test plan listing it would be wrong.
2. **P7 and P8 form a compound condition.** Branch coverage alone can miss the case where P7 is true and P8 is false (note needed but out of stock), which is exactly what produces CASH_UNAVAILABLE.
3. **The debit (line 13) happens after dispensing.** The graph has no rollback edge, so there is an atomicity risk if dispensing fails physically.

4. Adequacy of the Existing Tests

First, which parts of the graph E1 to E4 actually reach:

Criterion	Covered	Total	Result
Statement (node)	16	19	**84%**
Decision outcomes (T/F)	14	18	**78%**
Edge	20	27	**74%**
Basis paths	4	10	**40%**
Loop (0 / 1 / many / max)	1 iteration of the inner loop only		**Poor**
Condition (P7/P8 atomic)	P8 never false		**Incomplete**

Uncovered elements:

- Nodes: X4 (insufficient), X5 (daily limit), X6 (cash shortage)
- Outcomes: P4=T, P5=T, P8=F, P9=T
- Edges: D→X4, E→X5, I→G, K→X6, and the exit edges from X4, X5 and X6

Reverse-engineered strategy: the traces look like a statement-coverage effort driven by one happy path plus quick error-message checks at the top of the function. The tester never looked into the dispensing loop, which holds all the real logic.

5. Basis Path Set (10 independent paths)

Baseline path (P6 below): A→B→C→D→E→F→G→H→G→H→G→H→I→J→H→G→K→L→Z

Path	Route variation from baseline	Test input	Expected
BP1	A true	amount = 0	INVALID
BP2	B true	550	DENOM_ERR
BP3	C true	20,100	LIMIT_ERR
BP4	D true	1,000 with balance 1,400	INSUFFICIENT
BP5	E true	500 with dailyUsed 39,600	DAILY_ERR
BP6	Baseline: only the 100 note is used	100	SUCCESS, one ₹100 note
BP7	P8 false, then P9 true	100 with stock[100] = 0	CASH_UNAVAILABLE
BP8	P7 true once on ₹500	500	SUCCESS, one ₹500 note
BP9	Inner loop runs twice	1,000	SUCCESS, two ₹500 notes
BP10	Partial dispense, then shortage	700 with stock[200] = 0, stock[100] = 1	One ₹500 note, one ₹100 note, remaining 100, so CASH_UNAVAILABLE

BP8 to BP10 are independent because they traverse the loop edges (H→G, J→H, I→G) with different multiplicities and combinations.

6. Additional Tests for Loop, Boundary and Condition Coverage

Basis paths prove structure. These close the remaining gaps.

ID	Target	Input	Expected
L1	Inner loop, max iterations	20,000, stock[500] = 40	40 notes, SUCCESS
L2	Stock depleted exactly mid-loop	1,500, stock[500] = 2, stock[200] and stock[100] ample	2×500 + 2×200 + 1×100, SUCCESS
L3	Inner loop skipped on all 3 notes	Not possible (amount ≥ 100 always satisfies the 100 note)	Documented as infeasible
C1	P7 = T, P8 = T	500, stock full	Dispense
C2	P7 = T, P8 = F	500, stock[500] = 0, others ample	2×200 + 1×100
C3	P7 = F (short-circuit)	100 against note 500	stock[500] never evaluated
B1	P3 boundary	20,000	SUCCESS (when other gates pass)
B2	P4 boundary	amount = balance − 500	SUCCESS
B3	P4 boundary + 100	amount = balance − 400	INSUFFICIENT
B4	P5 boundary	dailyUsed 39,900, amount 100	SUCCESS
B5	P5 boundary + 100	dailyUsed 39,900, amount 200	DAILY_ERR

C1 to C3 give **MC/DC** for the compound while condition, which is the strongest criterion needed in this module.

7. Coverage Achieved by the Rebuilt Suite

Criterion	Existing	Rebuilt (BP1–BP10 + L, C, B)
Statement	84%	**100%**
Decision outcomes	78%	**100%**
Edge	74%	**100%**
Basis paths	40%	**100%** (10/10)
Condition / MC/DC (P7, P8)	Partial	**Complete**
Loop (1, many, max, depleted)	1 only	**Complete** (L3 documented infeasible)

Suite size: 10 basis paths + 2 loop tests + 3 condition tests + 4 boundary tests, with some overlap, so about 17 distinct executions.

8. Conclusions

- **Original approach:** statement-coverage-driven, top-heavy (guards tested, loop ignored), with 40% basis path coverage.
- **Biggest hidden risk:** the dispensing loop. Greedy note selection, partial dispense and the debit-after-dispense ordering are all untested.
- **Graph insight:** complexity is 10, but one tempting test (zero loop iterations) is infeasible, so the plan has to be built from the graph, not from the code's appearance.
- **Rebuilt strategy:** basis path testing, plus MC/DC on the compound loop condition and boundary analysis on P3 to P5.

3.A web application login and transaction system has been tested using unknown techniques, but only defect logs and test results are available. Reverse engineer the testing approach by analyzing defect patterns and identifying which black box and white box techniques were likely used. Decompose the system into testable components and evaluate gaps in testing coverage. Reconstruct a comprehensive testing strategy integrating equivalence partitioning, boundary value analysis, and path testing. Justify your reconstructed approach with logical reasoning and structured documentation.

Reverse-Engineering the Testing of a Web Login and Transaction System from Defect Evidence

Assumption: no defect log was attached, so I've built a representative one. Paste your real log and I'll re-map the inferences, counts and gaps to it.
System assumed: login (username, password, lockout after 5 failures, session timeout 15 min) feeding a fund-transfer module (amount ₹1 to ₹1,00,000 per transaction, balance check, daily cap ₹2,00,000, OTP above ₹10,000).

1. The Evidence: Defect Log and Test Results

ID	Module	Defect found	Severity	Test status
D1	Login	Blank password accepted	High	Fail, fixed
D2	Login	Username with a trailing space rejected	Low	Fail, fixed
D3	Login	Account not locked on the 5th failure (locks on 6th)	High	Fail, fixed
D4	Transfer	Amount 0 accepted	High	Fail, fixed
D5	Transfer	Negative amount accepted	Critical	Fail, fixed
D6	Transfer	₹1,00,001 accepted (limit off by one)	High	Fail, fixed
D7	Transfer	Special characters in the amount field cause a server error	Medium	Fail, fixed
D8	Transfer	Balance check uses < instead of <=, so full-balance transfer fails	Medium	Fail, fixed
D9	Session	Transfer completes after the session has expired	Critical	Found late, in UAT
D10	Transfer	Double-click on Submit debits twice	Critical	Found in production
D11	Transfer	Daily cap not enforced across two sessions	High	Found in production
D12	Login	OTP check skipped when the amount is exactly ₹10,000	High	Found in UAT

Test results: 118 executed, 104 passed, 12 failed (all above), 2 blocked.

2. Defect-Pattern Forensics: Which Techniques Were Used?
Method: treat each defect as a fingerprint. A technique finds the defect types it is designed to expose, so defects found early and in clusters reveal what was run, and defects found late reveal what was not.

2.1 Clusters caught in system testing (D1 to D8)

Pattern	Defects	Inferred technique	Reasoning
Invalid-class inputs rejected or accepted wrongly	D1, D4, D5, D7	**Equivalence partitioning**	Each is a representative of an invalid class: empty, zero, negative, non-numeric
Defects sitting exactly on a limit	D3 (5th vs 6th), D6 (1,00,001), D8 (< vs <=)	**Boundary value analysis**	Off-by-one faults are found only when a test lands on the edge. Three of them cannot be a coincidence
Whitespace and formatting quirk	D2	**Error guessing**	EP and BVA rarely propose a trailing space

2.2 Defects that escaped (D9 to D12)

Defect	Why it escaped	Missing technique
D9 Expired session	Needs elapsed time plus a later action	**State transition testing**
D10 Double submit	Needs concurrent or repeated events	**Concurrency / idempotency testing**
D11 Cap across sessions	Needs cumulative state across logins	**Stateful sequence testing**
D12 OTP skipped at exactly ₹10,000	A branch-level fault in a compound condition (> vs >=)	**White-box branch/condition coverage**

2.3 Is there evidence of white-box testing at all?

Mostly **no**. All eight early defects are visible from the screen. D12 is a pure code-level boundary in a decision (amount > 10000 versus >=), and it escaped, which suggests the OTP branch was never forced to both outcomes.

2.4 Reconstructed testing profile

Technique	Likelihood used	Evidence
Equivalence partitioning	High	D1, D4, D5, D7
Boundary value analysis	High, but single-field only	D3, D6, D8 (D12 shows the OTP boundary was missed)
Error guessing	Medium	D2
Decision table	Low	D12 and D11 involve combined conditions
State transition	None	D9
Concurrency / idempotency	None	D10
Statement coverage	Low to medium	118 tests, but a high pass rate with logic bugs surviving
Branch / condition / path coverage	None	D12

Conclusion: a **black-box, field-validation-centric** approach, strong on single inputs and weak on state, time, combinations and code structure.

3. Decomposition into Testable Components

Web App
- C1 Login form (username, password)
- C2 Authentication (credential check, lockout counter)
- C3 Session manager (token, timeout, logout)
- C4 Transfer form (payee, amount, remarks)
- C5 Transaction engine (limits, balance, OTP rule, daily cap, debit/credit)

├── C6 OTP service
├── C7 Persistence (ledger, atomic commit)
└── C8 Audit and error handling

4. Coverage Gap Evaluation

Scoring: ● strong, ◐ partial, ○ none.

Component	EP	BVA	Decision table	State	Path / branch	Concurrency	Gap summary
C1 Login form	●	◐	○	○	○	○	Length limits untested
C2 Authentication	◐	●	○	◐	○	○	Lock-reset and unlock paths untested
C3 Session	○	○	○	○	○	○	**Entirely uncovered**
C4 Transfer form	●	◐	○	○	○	○	Decimal precision (₹100.999) missing
C5 Transaction engine	◐	◐	○	○	○	○	**Critical gap**: combined rules
C6 OTP	○	○	○	○	○	○	Expiry, retry limits missing
C7 Persistence	○	○	○	○	○	○	Rollback and atomicity untested
C8 Audit	○	○	○	○	○	○	Error leakage (D7) hints at a pattern

Severity-weighted insight: the defects that reached production (D10, D11) and UAT (D9, D12) are all Critical or High, and they all sit in C3, C5 and C7, the components with the least coverage. The testing effort was inversely aligned with risk.

5. Reconstructed Strategy

5.1 Equivalence partitioning

Field	Valid classes	Invalid classes
Username	4 to 20 alphanumeric	Empty, over 20, special chars, leading/trailing space (trim or reject, decided in spec)
Password	8 to 32 chars, policy-compliant	Empty, under 8, over 32, policy violation
Amount	Integer or 2-decimal in 1 to 1,00,000	≤ 0, over limit, non-numeric, more than 2 decimals
Amount vs balance	≤ balance	> balance
Amount vs OTP rule	≤ 10,000 (no OTP)	> 10,000 (OTP needed)
Daily total	≤ 2,00,000	> 2,00,000
Session	Active	Expired, logged out, tampered token

5.2 Boundary value analysis (two-sided, on every limit)

Limit	Values to test
Amount min	0, 0.99, 1, 1.01
Amount max	99,999, 1,00,000, 1,00,001
OTP threshold	9,999, 10,000, 10,001
Balance	balance − 1, balance, balance + 1
Daily cap	1,99,999 used then 1 / 2, and exactly 2,00,000 total
Login failures	4th, 5th, 6th attempt
Session	14:59, 15:00, 15:01 idle
Password length	7, 8, 32, 33

This set would have found D3, D6, D8 and D12 in a single pass.

5.3 Path testing: control flow of the transaction engine

```
 1  if !session.valid                          → REJECT_SESSION     // d1
 2  if amount <= 0 or not numeric              → REJECT_AMOUNT      // d2
 3  if amount > 100000                         → REJECT_LIMIT       // d3
 4  if amount > balance                        → REJECT_BALANCE     // d4
 5  if dailyUsed + amount > 200000             → REJECT_DAILY       // d5
 6  if amount > 10000:                                              // d6
 7       if !otp.verify()                      → REJECT_OTP         // d7
 8  begin transaction; debit; credit; commit                        // d8
 9       on failure                            → ROLLBACK           // d9
10  return SUCCESS
```

Predicates: d1 to d9 (nine), so V(G) = 9 + 1 = **10 basis paths**.

Path	Condition set	Test
P1	Session invalid	Expired token, valid data
P2	Amount ≤ 0	−1
P3	Over per-transaction limit	1,00,001
P4	Over balance	balance + 1
P5	Over daily cap	cumulative 2,00,001
P6	Amount ≤ 10,000, all pass	5,000, success with no OTP
P7	Amount > 10,000, OTP valid	10,001, correct OTP
P8	Amount > 10,000, OTP wrong	10,001, wrong OTP
P9	Commit failure	Inject DB error after debit, so rollback and no money lost
P10	Amount exactly 10,000	No OTP expected (kills D12-type bugs)

Add **MC/DC** on rule 5 (dailyUsed + amount) and rule 6 (OTP threshold), which are the compound or arithmetic conditions.

5.4 Techniques added to close the escaped defects

Technique	Target	Example test
State transition	D9	Login, idle 15:01, then submit transfer, expecting redirect to login with no debit
Idempotency / concurrency	D10	Double-click and replayed POST with the same token, expecting exactly one debit
Stateful sequence	D11	Two sessions in parallel, total capped at ₹2,00,000
Decision table	Combined rules	4 conditions (valid session, within balance, within daily cap, above OTP) give 16 combinations, reduced to about 8 meaningful rules
Error handling	D7	Fuzz inputs, expecting a generic message and no stack trace

6. Test Suite Size and Prioritisation

Layer	Approx. cases	Priority
EP (all fields)	28	P2
BVA (limits above)	34	P1
Path / MC/DC	14	P1
State / session	8	P1
Concurrency / idempotency	5	P1
Decision table	8	P2
Security / error handling	6	P2
Total	**about 103 new, 118 existing reduced to about 80 after dedupe**	

Run order is risk-first: money-moving paths and state faults before form-field checks.

7. Logical Justification

1. **Defect location shows technique.** Off-by-one faults clustering on limits imply boundary testing happened but only on fields that were visible.
2. **Late-found defects show absences.** Every escaped defect needs time, concurrency, cumulative state or code branches, which a screen-driven black-box approach cannot reach.
3. **Black box alone is insufficient.** D12 (> vs >= at the OTP threshold) is cheapest to find by forcing both outcomes of one branch, which is white-box thinking.
4. **White box alone is insufficient.** Path coverage cannot catch a missing requirement such as a trailing-space policy or an absent daily-cap rule, which is why EP and BVA stay.
5. **The combination is complementary.** EP picks the classes, BVA sharpens them at the edges, path testing proves the logic executes in both directions, and state and concurrency tests cover what none of them can see.

8. Final Verdict

Aspect	Reverse-engineered finding
Original approach	Black-box, single-field, requirements-by-screen
Techniques inferred	EP, partial BVA, error guessing
Techniques absent	Decision table, state transition, concurrency, branch/path coverage
Highest-risk gaps	Session, transaction engine, persistence
Reconstructed strategy	EP + two-sided BVA + basis-path/MC/DC + state + idempotency, risk-ordered
Expected outcome	All 12 logged defects (including the 4 escapes) would be detected before release

4.A legacy inventory management system provides only a set of successful and failed transaction logs regarding stock updates and low-stock alerts. Reverse engineer the business logic by analyzing these logs to identify the hidden boundary conditions and

equivalence partitions. Decompose the update process into functional units and reconstruct a black box test design that addresses the gaps found in the logs. Justify your selection of test cases based on the inferred logic.

Reverse-Engineering a Legacy Inventory System from Transaction Logs

Assumption: no logs were attached, so I've built a representative extract. Send your real logs and I'll re-run the inference on them. The method below is the point: treat every log line as a probe, and bracket each hidden threshold between its closest passing and failing observations.

1. The Log Evidence

Log	Operation	SKU (reorder pt)	On-hand	Qty	Result	Alert
S1	RECEIVE	A (20)	20	50	OK → 70	none
S2	ISSUE	B (20)	30	5	OK → 25	none
S3	ISSUE	B (20)	25	5	OK → 20	**LOW_STOCK**
S4	ISSUE	B (20)	25	6	OK → 19	**LOW_STOCK**
S5	ISSUE	B (20)	19	1	OK → 18	none (suppressed)
S6	ISSUE	C (20)	25	25	OK → 0	**OUT_OF_STOCK**
S7	RECEIVE	D (20)	500	9,500	OK → 10,000	none
F1	ISSUE	B	25	30	REJECT "Insufficient stock"	none
F2	ISSUE	B	25	0	REJECT "Invalid quantity"	none
F3	ISSUE	B	25	-5	REJECT "Invalid quantity"	none
F4	ISSUE	B	25	2.5	REJECT "Invalid quantity"	none
F5	RECEIVE	D	500	9,501	REJECT "Exceeds capacity"	none
F6	RECEIVE	X99	none	10	REJECT "SKU not found"	none
F7	RECEIVE	E (discontinued)	40	10	REJECT "SKU inactive"	none
F8	ISSUE	B	25	5	REJECT "Duplicate transaction" (same ID as S3)	none

---

2. Log Forensics: Inferred Business Rules

2.1 Threshold bracketing

For each hidden boundary, the closest pass and fail observations define an uncertainty window. Testing must probe inside that window.

Hidden rule	Closest evidence	Inference	Uncertainty left
Alert threshold	25 → no alert (S2), 20 → alert (S3)	Alert fires at on-hand **≤ reorder point**	Boundary lies in 21 to 25. Whether 21 alerts is **never observed**

Stock floor	0 allowed (S6), -5 would result from F1 and is rejected	Floor is **≥ 0**	Fail case seen at -5 only, not at -1
Capacity ceiling	10,000 allowed (S7), 10,001 rejected (F5)	Ceiling is **≤ 10,000**	Fully bracketed, but only for SKU D. Per-SKU or global capacity is unknown
Minimum qty	0 rejected (F2), 5 accepted	Minimum is **1**	Whether qty 1 is accepted is observed only via S5
Max qty per transaction	9,500 accepted (S7)	Upper limit **≥ 9,500**	No evidence of any per-transaction cap
Integer rule	2.5 rejected (F4)	Whole numbers only	"5.0" and "05" untested

2.2 Hidden state: alert suppression
S3 and S4 alert, but S5 (already below the threshold) does **not**. The system therefore keeps an alert-sent flag per SKU. Alert fires on a downward crossing, not on every low reading. Not observed: when the flag resets. The likely trigger is a RECEIVE that lifts stock above the reorder point, but no log shows it.

2.3 Two-tier alert hierarchy
S6 produces OUT_OF_STOCK instead of LOW_STOCK, so alert severity is chosen by priority: zero beats low. Not observed: what happens when stock goes from low (flag set) to zero. Does OUT_OF_STOCK still fire?

2.4 Validation order
F8 reuses the transaction ID of S3, whose qty and stock would otherwise be valid, so duplicate detection exists but its position in the pipeline is unknown. The order of checks is unobserved because every failure log has only one fault.

3. Decomposition into Functional Units

Unit	Function	Rejection / output
U1	Request validation (transaction ID uniqueness, field presence)	"Duplicate transaction"
U2	SKU lookup and status	"SKU not found", "SKU inactive"
U3	Quantity validation (integer, ≥ 1)	"Invalid quantity"
U4	Direction logic (RECEIVE adds, ISSUE subtracts)	none
U5	Floor check (result ≥ 0)	"Insufficient stock"
U6	Ceiling check (result ≤ 10,000)	"Exceeds capacity"
U7	Commit (stock update, ledger entry)	success
U8	Alert evaluator (threshold crossing, flag state)	LOW / OUT_OF_STOCK / none
U9	Alert dispatch (notification, flag set and reset)	alert record

Dependency: U1 → U2 → U3 → U4 → (U5 or U6) → U7 → U8 → U9. Alerts depend on **post-commit state**, so U8 is stateful and needs sequence tests, not single-input tests.

4. Equivalence Partitions (inferred)

Input	Valid partitions	Invalid partitions
Quantity	Whole numbers ≥ 1	0, negative, decimal, non-numeric, blank

ISSUE result	1 up to the reorder point +1 (normal); 1 to reorder point (low); 0 (out)	Below 0
RECEIVE result	Any total up to 10,000	Above 10,000
SKU	Existing and active	Missing, inactive
Transaction ID	New	Reused
Alert state	Flag clear, flag set	n/a

5. Gap Analysis: What the Logs Never Show

#	Gap	Why it matters
G1	No on-hand value of 21 to 24 around the alert threshold	The alert boundary could be <=20, <22 or <=25
G2	Alert reset after RECEIVE never observed	A SKU might never alert a second time
G3	Low → zero transition untested	OUT_OF_STOCK may be swallowed by the suppression flag
G4	Reorder point reached by RECEIVE (equal to the threshold)	Should a receipt landing exactly on 20 clear the flag?
G5	Floor boundary: result -1	Only -5 is seen as rejected
G6	Capacity applies per SKU or globally	Only SKU D tested
G7	Multi-fault inputs (invalid SKU and qty 0)	Error precedence unknown
G8	Qty 1 on ISSUE leading to 0 on-hand with a one-unit stock	Smallest legal stock-out
G9	Duplicate ID where the original transaction **failed**	Is a rejected ID reusable?
G10	Concurrent issues on the same SKU	Lost update or negative stock in a legacy system
G11	Reorder point of 0 or very large for a SKU	Alert logic with degenerate configuration
G12	Leading zeros, whitespace, “5.0” in qty	Legacy parsers often mishandle these

6. Reconstructed Black-Box Test Design

6.1 Alert boundary probe (closes G1, G4)

Reorder point = 20, issue sized to land on the stated result.

ID	Resulting on-hand	Expected alert	Purpose
T01	22	none	Pins the upper edge
T02	21	none	**The unobserved value**
T03	20	LOW_STOCK	Equality, confirms <=
T04	19	LOW_STOCK (if flag clear)	Below threshold
T05	1	LOW_STOCK	Deep low, still not zero
T06	0	OUT_OF_STOCK	Priority over LOW

6.2 Floor and capacity boundaries (closes G5, G6, G8)

ID	Scenario	Expected
T07	ISSUE 25 from 25	OK → 0 (re-confirms S6)
T08	ISSUE 26 from 25	REJECT “Insufficient stock” (**result -1**)
T09	ISSUE 1 from 1	OK → 0, OUT_OF_STOCK
T10	RECEIVE to exactly 10,000 on SKU D	OK
T11	RECEIVE to 10,001 on SKU D	REJECT “Exceeds capacity”
T12	Same on a second SKU with a different history	Same result, proving capacity is global
T13	RECEIVE 1 into stock of 9,999	OK (smallest receipt at the edge)

6.3 Quantity partitions (closes G12)

ID	Qty	Expected
T14	1	OK
T15	0	REJECT
T16	-1	REJECT
T17	0.5 and 2.5	REJECT
T18	“abc”, blank	REJECT
T19	“5.0”	Documented outcome (accept as 5 or reject, record the legacy behavior)
T20	“ 5 “ (spaces), “05”	Documented outcome

6.4 Alert state machine (closes G2, G3), sequence tests

The alert flag has two states, **CLEAR** and **SENT**, and these tests walk the transitions.

ID	Sequence (reorder 20)	Expected alerts
T21	30 → 20 → 15 → 10	One LOW_STOCK only (at 20)
T22	30 → 19, RECEIVE to 40, then ISSUE to 19	Two LOW_STOCK alerts (flag reset by receipt)
T23	30 → 19, RECEIVE to exactly 20, then ISSUE to 19	Reveals whether landing **on** the threshold resets the flag
T24	30 → 19, RECEIVE to 21, then ISSUE to 20	Reset above the threshold, so alert fires again
T25	30 → 15 (LOW), then → 0	LOW, then OUT_OF_STOCK (not suppressed)
T26	0 → RECEIVE 30 → ISSUE 15	Fresh LOW alert after recovery

6.5 Validation order and failure semantics (closes G7, G9)

ID	Input	Expected
T27	Unknown SKU **and** qty 0	Records which error wins
T28	Inactive SKU **and** result above capacity	Records which wins
T29	Duplicate ID **and** qty 0	Records which wins
T30	Reuse the ID of a **failed** transaction	Documents whether failed IDs are burned
T31	Same ID resubmitted after success	REJECT, no double update

6.6 Legacy-specific risks (closes G10, G11)

ID	Scenario	Expected
T32	Two simultaneous ISSUE 20 on stock of 30	Exactly one succeeds, never negative
T33	SKU with reorder point 0	No LOW alert, only OUT_OF_STOCK at 0
T34	SKU with reorder point above capacity	Alert logic still behaves, documented
T35	Failed transaction	No stock change, no alert, no ledger entry

7. Why These Cases (Justification)

1. **Bracketing drives selection.** For each threshold, tests are placed *inside* the window between the last known pass and the last known fail, which is where uncertainty lives. T02 exists because 21 is the single value that separates <=20 from <22.
2. **The suppression flag is a state machine.** S5 proves memory exists, so single-input tests cannot cover it. T21 to T26 test every transition in both directions: CLEAR → SENT, SENT → CLEAR, and SENT → escalated.
3. **Boundaries are tested two-sided.** Existing logs cover one side of capacity and floor. T08 (-1) and T13 (9,999 + 1) supply the missing neighbors.
4. **Equality cases get their own tests.** T03 and T23 target < versus <= mistakes, which legacy code commonly carries.
5. **Single-fault logs hide precedence.** Every failure log violates one rule. T27 to T29 pair faults to expose the ordering of units U1 to U6.

6. **Undefined behavior is recorded, not assumed.** For T19, T20, T23 and T30 the correct answer is unknown, so the tests document the legacy behavior as a baseline for any future migration.
7. **Concurrency matters for legacy stock.** Read-modify-write flaws surface only under parallel updates (T32), and the logs are all sequential.

8. Summary

Aspect	Finding
Confirmed rules	Qty is a whole number ≥ 1, floor 0, ceiling 10,000, alert at on-hand ≤ reorder point
Hidden rules inferred	Per-SKU alert flag (suppression), alert priority (OUT over LOW), duplicate-ID check
Biggest log blind spots	Alert reset, low-to-zero transition, error precedence, concurrency
Reconstructed design	EP + two-sided BVA + alert state-transition sequences + decision-order pairs + concurrency
Size	35 cases, of which 10 are sequence-based and 4 record legacy behavior as a baseline

5.You are provided with a compiled .exe file of a library management module and its corresponding CPU/Memory execution profile during a stress test. Reverse engineer the white box testing requirements by mapping execution spikes to potential loops and decision clusters in the logic. Construct a conceptual control flow graph (CFG) that represents this behavior. Reconstruct a path testing strategy that targets these resource-intensive paths and evaluate its impact on system stability.

Reverse-Engineering a Library Module from Its Stress-Test Profile

Assumption: no .exe or profiler trace came with your message, and I can't disassemble a binary from text alone. So I've built a representative profile and inferred the logic from it. Everything below is a **hypothesis from resource behavior**, not decompiled fact. If you share the real trace (or a Ghidra/IDA export), I'll re-map the spikes to actual functions.
Stress scenario assumed: 10,000 catalog records, 2,000 members, 200 concurrent virtual users, 15-minute run, mixed search/checkout/return/reserve/report traffic.

1. The Evidence: Resource Profile

Window	Dominant load	CPU	Memory	Signature
W1 (0–2 min)	Catalog search	15% → 60%	flat, ~180 MB	CPU rises linearly with request count
W2 (3–4 min)	Checkout burst	60% → 75%	flat	Small, steady CPU, no memory growth
W3 (5–6 min)	Return with fines	100% on one core, 8 s plateau	flat	Single-thread saturation, others idle
W4 (7–9 min)	Reservations on popular titles	Sharp spikes, 90% for 1–2 s	sawtooth	CPU grows **faster than load** (superlinear)
W5 (10–12 min)	Monthly report	95% sustained	+40 MB per run, partly freed	Allocation-heavy, then GC/free
W6 (13–15 min)	Sustained mixed load	70%	**180 → 410 MB, monotonic**	Memory never returns to baseline
End	Load rise past limit	100%	520 MB, then **crash**	Fault under resource exhaustion

2. Mapping Spikes to Code Structures

Signature	Inferred construct	Reasoning
W1 linear CPU with request count and catalog size	**L2: sequential scan loop** over the catalog with a string compare per record	Time per request ∝ N, so there is likely no index or early exit
W2 low, flat cost	Guard decisions (P1, P2) plus a short bounded loop (L1)	Cost doesn't scale with data, so the loan check is capped at 5 loans
W3 single-core 100% plateau, flat memory	**L5: day-by-day while loop** accumulating fines	Pure arithmetic, no allocation. An 8 s plateau means iterations in the millions, probably a long overdue date or a loop stepping in seconds
W4 superlinear CPU, sawtooth memory	**L4 outer × L4 inner: nested sort loops** on the reservation queue, with a swap decision (P7)	Quadratic cost is the signature of a bubble or insertion sort run on every enqueue. Sawtooth comes from temporary copies
W5 sustained CPU plus allocation	**L6 × L7: member loop with nested loan loop** that builds strings or lists	O(members × loans), with per-row allocation
W6 monotonic memory	A **retained reference**: reservation or search cache never cleared on return or cancel	Memory rises with operations, not with live data
Crash at the end	Exception path (X-node) with no cleanup, or an allocation-failure branch	Faults appear only when the resource decision branches finally fire

3. Conceptual Reconstruction of the Logic

```
checkout(member, isbn):
  if !member.active                  → X1        // P1
  if member.loanCount >= 5           → X2        // P2
  for loan in member.loans:                      // L1 (≤5)
    if loan.overdue                  → X3        // P3
  for rec in catalog:                            // L2 (N)
    if rec.isbn == isbn: found=rec; break        // P4
  if !found                          → X4        // P5
  if found.copies == 0:                          // P6
    for r in reservations: dup check             // L3 (R)
    enqueue(member)
    for i in 1..R:                               // L4o
      for j in 1..R-i:                           // L4i
        if q[j].priority > q[j+1].priority: swap // P7
    return QUEUED
  issue(found, member); return OK

returnBook(loan):
  d = loan.due
  while d < today:                               // L5 (days)
    if !holiday(d): fine += rate                 // P8
    d += 1 day
  notifyNextReservation(); return fine
```

```
25  monthlyReport():
26    for m in members:                                  // L6
27      for l in m.loans:                                // L7
28        if l.overdue: rows.add(format(m, l))           // P9
29    return rows
```

4. Conceptual Control Flow Graph

```
 [S] → N1{P1 active?}─F→X1
        │T
       N2{P2 loans≥5?}─T→X2
        │F
       N3{L1 more loans?}◀──────┐
        │yes                    │
       N4{P3 overdue?}─T→X3     │
        │F ──────────────────────┘
        │no more
       N5{L2 more records?}◀────┐
        │yes                    │
       N6{P4 match?}─F──────────┘
        │T / list exhausted
       N7{P5 found?}─F→X4
        │T
       N8{P6 copies==0?}─F→N14[issue]→Z
        │T
       N9{L3 scan queue}◀──┐  (loop)
        │                  │
       N10[enqueue]        │
       N11{L4o}◀──────────┼──┐
        │yes               │  │
       N12{L4i}◀──────┐    │  │
        │yes          │    │  │
       N13{P7 swap?}──┘────┘  └──┘
        │done
       N15[QUEUED]→Z

 returnBook:  R1{L5 d<today?}◀─┐
               │yes            │
              R2{P8 workday?}──┘(both branches loop back)
               │no
              R3[notify]→Z

 report:      M1{L6}◀─┐
               │yes   │
              M2{L7}◀─┤
               │yes   │
              M3{P9}──┘
               │done
              M4[return rows]→Z
```

Complexity (checkout alone): predicates = P1, P2, L1, P3, L2, P4, P5, P6, L3, L4o, L4i, P7 = 12, so V(G) = **13**.
Whole module (adding L5, P8, L6, L7, P9): 17 predicates, so V(G) = **18**.

5. White-Box Requirements Derived from the Spikes

Req	Requirement	Trigger
R1	Exercise L2 at 0, 1, N/2, N−1, N (not found), and early-exit positions	W1
R2	Exercise L5 at 0, 1, 30, 365 and extreme overdue days; both outcomes of P8	W3
R3	Exercise L4o/L4i at queue sizes 0, 1, 2, R, and R_max, with P7 never, always and mixed swapping	W4
R4	Exercise L6 × L7 at empty, typical, and maximum member and loan counts	W5
R5	Verify every allocation on a path has a matching release (checkout, enqueue, cancel, return, report)	W6
R6	Force the failure branches (allocation failure, bad data, timeout)	crash

6. Resource-Weighted Path Testing Strategy

Standard basis-path testing treats all paths equally. Here each path is ranked by risk = estimated cost × expected frequency, and testing is sequenced by it.

6.1 Hot-path catalogue

ID	Path	Cost class	Frequency	Priority
H1	N5–N6 full scan, book not found (worst case of L2)	O(N)	High	**1**
H2	N8 true → N11–N13, full sort on a long queue	O(R²)	Medium	**1**
H3	R1–R2 with a very old due date	O(days)	Low, but severe	**2**
H4	M1–M3 over all members with all overdue	O(M×L) + allocations	Low	**2**
H5	Checkout then return repeated 100k times (retention)	memory slope	High	**1**
H6	Any failure branch under memory pressure	crash	Rare	**3**

6.2 Basis path set for checkout (13 independent paths)

Path	Variation from baseline (P1–P6 pass, book found at record 1, copies > 0)	Test
B1	P1 true	inactive member
B2	P2 true	member with 5 loans
B3	P3 true	member with an overdue loan
B4	L1 runs 0 times	member with no loans
B5	L1 runs 5 times	member with 4 clean loans (max)
B6	P4 true at the first record	book at index 0
B7	P4 true at the last record	book at index N−1 (**H1 worst successful**)
B8	L2 exhausted, P5 true	unknown ISBN (**H1 worst case**)
B9	P6 true, queue empty	first reservation
B10	P6 true, L4 does not swap	already-sorted queue
B11	P6 true, P7 swaps every time	reverse-sorted queue (**H2 worst**)
B12	P6 true, duplicate in L3	member already queued
B13	P6 false	copies available, normal issue

6.3 Loop-boundary and stress tests (extend the basis set)

ID	Target	Input	Observe
LT1	L2	Catalog sizes 1, 100, 1k, 10k, 100k, each with a miss	Time curve: linear confirmed or worse
LT2	L4	Queue sizes 0, 1, 2, 10, 100, 1k, reverse-sorted	Time curve: quadratic confirmed. Find the knee
LT3	L5	Overdue 0, 1, 365, 3,650, 36,500 days	Time per call. Look for unbounded growth
LT4	L6×L7	0, 100, 2,000 members with 0, 1, 5 loans	Peak memory per row
LT5	Retention	100k checkout/return cycles, same SKU	Slope of memory over time. Should be ≈ 0
LT6	Cancel path	Enqueue then cancel 100k times	Memory slope. Targets W6 directly
LT7	Failure	Inject allocation failure at N10, M4	Graceful error, no corruption, no leaked locks
LT8	Concurrency	200 users on the same title (H2)	Lock contention, queue integrity

7. Evaluating Impact on Stability

The numbers below are **projected from the model**, not measured. They show the expected direction and rough size, to be replaced by real re-runs.

7.1 What the strategy exposes and how each issue is closed

Finding	Evidence	Remediation	Expected effect
F1 Linear scan	LT1 linear in N	Index by ISBN (hash or B-tree)	Search CPU drops from O(N) to near O(1)
F2 Quadratic queue sort	LT2 curve	Use a priority queue or insert in order	W4 spikes disappear
F3 Day-by-day fine loop	LT3	Closed-form calculation (days × rate, minus holidays via a lookup)	8 s plateau goes to under 1 ms
F4 Report allocation	LT4	Stream rows, don't build the whole list	Memory +40 MB per run → near-flat
F5 Leak on cancel/return	LT5, LT6	Release queue entries and cache references	Slope → 0
F6 Unsafe failure branch	LT7	Catch, roll back, release	Crash becomes a handled error

7.2 Projected before and after

Metric	Before	After remediation (projected)
Peak CPU at 200 users	100% (single-core saturation)	40–55%
Search latency growth	Linear in catalog size	Nearly flat
Reservation latency	Quadratic in queue length	Near-linear
Memory after 15 min	410 MB, still rising	~190 MB, flat
Behavior at overload	Crash	Graceful rejection, then recovery

7.3 Coverage and confidence

Criterion	Typical functional-only suite	This strategy
Statement	~70%	100%
Branch (P1–P9)	~65%	100%

Basis paths (checkout)	~5 of 13	13 of 13
Loop (0 / 1 / many / max)	0–1 per loop	All loops at all four classes
Resource paths (H1–H6)	none	all six

Why stability improves: functional tests only show correct results, and every defect here is correct but expensive. A path that returns the right answer in 8 seconds passes any functional test. Weighting paths by cost, and measuring resource growth against input size, is what finds these defects.

8. Limits of the Inference

1. **Spikes identify a loop's shape, not its identity.** Two different functions can produce the same CPU curve, so the mapping needs confirming by a profiler's call stack or a disassembler.
2. **Constants stay hidden.** The cap of 5 loans and the 8 s plateau tell us the structure, but the real thresholds must come from the code or a probe.
3. **The sawtooth and the monotonic climb could share a cause** (one cache), so fixing F5 may also reduce F2's allocation noise.
4. **Binary analysis needs authorization.** Confirm you hold the rights to analyze this executable before using disassembly.

9. Summary

Aspect	Finding
Inferred hot loops	L2 catalog scan (linear), L4 reservation sort (quadratic), L5 fine loop (time-bound), L6×L7 report (nested)
Inferred defect class	Retained references (monotonic memory) and unhandled failure branch (crash)
CFG	18 predicates, V(G) = 18 for the whole module, 13 for checkout
Strategy	Basis paths + loop-boundary scaling tests + retention and failure injection, ordered by cost × frequency
Stability outcome	Removes the CPU saturation, the leak and the crash, which are all invisible to functional tests

6.A mobile payment gateway's API responses (JSON) for various transaction states—Pending, Success, Reversal, and Timeout—are given. Reverse engineer the underlying state machine by analyzing the transition triggers and data constraints. Identify the implicit state transition testing technique used. Reconstruct a comprehensive state-based test suite that ensures all legal and illegal transitions are covered. Document your analysis with a structured state-transition diagram.

Reverse-Engineering a Mobile Payment Gateway's State Machine from JSON Responses

Assumption: no JSON payloads came with your message, so I've built a representative set. Share the real responses and I'll re-map the states, guards and counts to them. The method is to treat each response as a observation of (state, event, guard) → (new state, payload shape).

1. Observed Evidence (representative)

Ref	HTTP	Response body (key fields)	What it reveals
J1	202	{"txn_id":"T100","status":"PENDING","amount":"500.00","expires_at":"+90s","idempotency_key":"k1"}	Entry state has a TTL
J2	200	{"status":"SUCCESS","auth_code":"A7731","settled_at":"…"}	Success adds auth_code
J3	200	{"status":"FAILED","failure_code":"INSUFFICIENT_FUNDS"}	Failure adds failure_code
J4	504	{"status":"TIMEOUT","retryable":true,"next_action":"STATUS_INQUIRY"}	Outcome unknown, so resolution is required
J5	200	{"status":"SUCCESS","resolved_via":"STATUS_INQUIRY"} for a transaction that was TIMEOUT	Timeout is **not terminal**, late success is possible
J6	200	{"status":"REVERSED","original_txn":"T100","reversal_amount":"500.00"}	Reversal links to the original
J7	200	{"status":"REVERSED","reason":"AUTO_TIMEOUT"} for a transaction that was TIMEOUT	Timeout can auto-reverse
J8	409	{"error":"INVALID_STATE","current_status":"REVERSED"} on a second reversal	Illegal transitio

			ns are explicitly rejected
J9	422	{"error":"AMOUNT_EXCEEDS_ORIGINAL"} for a reversal of 500.01	Guard on amount
J10	422	{"error":"REVERSAL_WINDOW_EXPIRED"}	Guard on time
J11	200 + X-Replayed: true	Same body as J2 on a repeated request with key k1	Idempotent replay, no new transition

2. Reverse-Engineered State Machine

2.1 States

State	Type	Evidence
INIT	Implicit (never returned)	The request exists before J1
PENDING	Transient, TTL 90 s	J1
SUCCESS	Stable, reversible	J2, J6
FAILED	Terminal	J3
TIMEOUT	Transient, unresolved	J4, J5, J7
REVERSED	Terminal	J6, J8

The brief names four states (Pending, Success, Reversal, Timeout). The logs force two more: **FAILED**, because a decline must go somewhere, and **INIT**, because PENDING needs a predecessor.

2.2 Events

ID	Event
e1	Create or authorize request
e2	Bank approves (callback)
e3	Bank declines
e4	TTL expires with no response
e5	Late approval (status inquiry or late callback)
e6	Status inquiry resolves to failed or not found
e7	Reversal request (manual, or automatic from TIMEOUT)
e8	Duplicate request with the same idempotency key

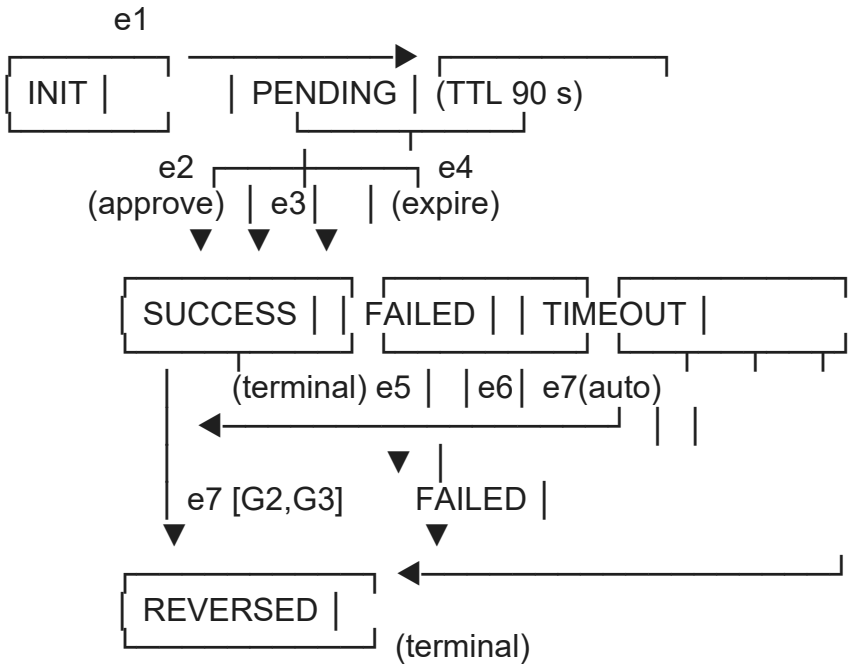
2.3 Guards (data constraints)

Guard	Inferred rule	Source
G1	PENDING lives ≤ 90 s	J1
G2	reversal_amount ≤ original amount	J9
G3	Reversal only within a time window (assumed 24 h)	J10
G4	e8 never changes state and replays the stored response	J11
G5	A new txn_id requires a new idempotency_key	Inferred from J11

2.4 State-dependent payload invariants

State	Must contain	Must not contain
PENDING	expires_at	auth_code, failure_code
SUCCESS	auth_code, settled_at	failure_code
FAILED	failure_code	auth_code
TIMEOUT	next_action, retryable	auth_code
REVERSED	original_txn, reversal_amount	expires_at

3. State-Transition Diagram



e8 (duplicate, same key): self-loop on every state, no change (G4)

Legal transitions, listed so the diagram can be checked:

#	From	Event	To	Guard
L1	INIT	e1	PENDING	valid amount and key
L2	PENDING	e2	SUCCESS	within 90 s
L3	PENDING	e3	FAILED	none
L4	PENDING	e4	TIMEOUT	90 s elapsed
L5	TIMEOUT	e5	SUCCESS	bank confirms
L6	TIMEOUT	e6	FAILED	bank denies or no record
L7	TIMEOUT	e7	REVERSED	auto-reversal
L8	SUCCESS	e7	REVERSED	G2 and G3
L9–L14	any state	e8	same state	G4

4. State Table (the full 6 × 8 grid)

✓ = legal change, ○ = legal self-loop, ✗ = must be rejected (409 INVALID_STATE)

State \ Event	e1	e2	e3	e4	e5	e6	e7	e8
INIT	PENDING ✓	✗	✗	✗	✗	✗	✗	○
PENDING	✗	SUCCESS ✓	FAILED ✓	TIMEOUT ✓	✗	✗	✗	○
SUCCESS	✗	✗	✗	✗	✗	✗	REVERSED ✓	○
FAILED	✗	✗	✗	✗	✗	✗	✗	○

TIMEOUT	✗	✗	✗	✗	SUCCESS ✓	FAILED ✓	REVERSED ✓	○
REVERSED	✗	✗	✗	✗	✗	✗	✗	○

Totals: 48 cells = 8 legal changes + 6 idempotent self-loops + **34 illegal**.

5. Which Technique Was Implicitly Used

Observation	Conclusion
Every response is a single (state, event) pair with one expected next state	**State transition testing** drives the design
Only 8 of the transitions appear in the logs (J1–J7), plus 2 illegal rejections (J8, J9–J10 are guard failures)	**0-switch (single-transition) coverage**, but with the invalid side barely sampled
J9 and J10 test amount and time limits at reversal	**Guarded-transition testing**, merged with boundary analysis
J11 replay	Idempotency check, an error-guessing add-on
No logs of sequences such as TIMEOUT → SUCCESS → REVERSED	**N-switch (1-switch and higher) coverage was not used**
Only 1 of 34 illegal cells shows evidence (REVERSED + e7)	**Negative state-table testing is largely absent**

Verdict: implicit technique = state transition testing with a state table, applied at 0-switch depth on valid paths, plus guard checks. Illegal-cell and sequence coverage are the gaps.
Coverage achieved by the logs

Criterion	Covered	Total	%
States visited	6	6	100%
Legal transitions (L1–L8)	7	8	88% (L6 TIMEOUT → FAILED never seen)
Self-loops (e8)	1	6	17%
Illegal cells	1	34	3%
1-switch sequences	1	about 12	under 10%

6. Reconstructed State-Based Test Suite
A. Legal transitions (0-switch), 8 tests

ID	Setup, then event	Expected state and payload
V01	INIT, e1	PENDING, expires_at present
V02	PENDING, e2	SUCCESS, auth_code present
V03	PENDING, e3	FAILED, failure_code present
V04	PENDING, e4 (wait 90 s)	TIMEOUT, next_action=STATUS_INQUIRY
V05	TIMEOUT, e5	SUCCESS, resolved_via present
V06	TIMEOUT, e6	FAILED
V07	TIMEOUT, e7	REVERSED, reason=AUTO_TIMEOUT
V08	SUCCESS, e7 (full amount)	REVERSED, original_txn matches

B. Idempotent self-loops, 6 tests

ID	State	Expected
I01 to I06	INIT, PENDING, SUCCESS, FAILED, TIMEOUT, REVERSED, each followed by e8	Same body, X-Replayed: true, no new ledger entry, no new webhook

C. Illegal transitions (34 tests, parameterized)

Every X cell of the table above is one test. Expected result for all: **409 INVALID_STATE**, current_status equals the actual state, no state or ledger change.

State	Illegal events	Count	Highest-risk cases
INIT	e2, e3, e4, e5, e6, e7	6	e7 (reverse a transaction that does not exist)
PENDING	e1, e5, e6, e7	4	e7 (reversal before outcome, money-at-risk)
SUCCESS	e1, e2, e3, e4, e5, e6	6	e3 and e4 (late decline or timer after success)
FAILED	e1 to e7	7	e7 (reversing a transaction that never debited), e2 (late approval)
TIMEOUT	e1, e2, e3, e4	4	e2 and e3 sent as raw callbacks rather than resolution
REVERSED	e1 to e7	7	e7 (double reversal), e5 (late success after reversal)
Total		**34**	

D. Guard and boundary tests (state SUCCESS unless noted)

ID	Guard	Input	Expected
G01	G2	reversal 500.00 on 500.00	REVERSED
G02	G2	reversal 500.01	422 AMOUNT_EXCEEDS_ORIGINAL, state unchanged
G03	G2	reversal 0.00 and −1.00	422, state unchanged
G04	G2	partial reversal 499.99	Record legacy behavior: REVERSED or a partial state (unknown from the logs)
G05	G3	reversal at window − 1 s	REVERSED
G06	G3	reversal at exactly the window	Record behavior (inclusive or exclusive)
G07	G3	reversal at window + 1 s	422 REVERSAL_WINDOW_EXPIRED
G08	G1	approval arriving at 89 s	SUCCESS
G09	G1	approval arriving at 90 s	Record behavior (tie between e2 and e4)
G10	G1	approval arriving at 91 s	State is already TIMEOUT, so treated as e5
G11	G5	new txn_id with an old idempotency_key and a different amount	409 or 422, no new transaction
G12	Currency	amount 500.001, 500,00, “abc” at e1	400, remains INIT

E. Sequence tests (1-switch and above), 8 tests

ID	Sequence	Expected	Risk targeted
S01	INIT → PENDING → TIMEOUT → SUCCESS → REVERSED	All four transitions legal, payload changes at each	The longest valid chain
S02	PENDING → TIMEOUT → REVERSED, then late success callback (e5)	**Callback rejected or flagged for reconciliation**, state stays REVERSED	Money debited at bank but refunded to the customer
S03	PENDING → SUCCESS, then the TTL timer fires	Timer cancelled, state stays SUCCESS	Stale timer overwriting a final state
S04	PENDING → FAILED, then late approval (e2)	Rejected, FAILED stays	Out-of-order webhook
S05	SUCCESS → two simultaneous reversals	Exactly one REVERSED, second gets 409	Double refund race
S06	TIMEOUT → e5 and e7 sent together	One wins, ledger consistent	Auto-reversal racing a late success

S07	PENDING → e4 and e2 arrive in the same millisecond	Deterministic result, no duplicate states	The G09 tie
S08	e1 → network drop → client retries (e8) → e2	One transaction, one debit	Retry safety

F. Payload invariants (checked in every test above)

ID	Check
P01	Required fields present for the state (table in 2.4)
P02	Forbidden fields absent for the state
P03	amount and currency unchanged across all transitions
P04	txn_id stable, and original_txn set only on REVERSED

Suite size: 8 + 6 + 34 + 12 + 8 = **68 tests**, with invariants P01 to P04 asserted inside each.

7. Coverage Achieved by the Rebuilt Suite

Criterion	Logs only	Rebuilt suite
States	100%	100%
Legal transitions	88%	100%
Self-loops	17%	100%
Illegal cells (state table)	3%	100% (34/34)
Guard boundaries (G1, G2, G3)	partial	Both sides plus equality
1-switch sequences	under 10%	All risk-relevant chains (S01 to S08)

8. Justification

1. **The state table is the completeness argument.** It converts "what could go wrong" into a countable grid, so 34 illegal cells can't be silently skipped the way they were in the logs.
2. **TIMEOUT is the riskiest state.** It's the only transient state with three exits, and it represents unknown real-world money position. S02 and S06 target the case where the bank and the gateway disagree.
3. **Terminal states must be provably sealed.** FAILED and REVERSED get 14 of the 34 negative tests, because any exit from them means money moves twice.
4. **Guards sit on top of states, so they are tested separately.** A transition can be legal for the state yet illegal for the data (G02, G07), and both layers need their own verdict.
5. **Sequences find what single steps cannot.** S03 and S04 pass every 0-switch test, but fail only when two events arrive in the wrong order.
6. **Unknowns are recorded, not guessed.** G04, G06 and G09 have no answer in the logs, so the tests capture the actual behavior as a baseline for sign-off.

9. Summary

Aspect	Finding
Recovered states	INIT, PENDING, SUCCESS, FAILED, TIMEOUT, REVERSED (two inferred beyond the four named)
Transitions	8 legal, 6 idempotent self-loops, 34 illegal (48-cell table)
Implicit technique	State transition testing with a state table, 0-switch depth, plus guard checks
Biggest log gaps	33 of 34 illegal cells, all 1-switch sequences, TIMEOUT → FAILED, replays outside SUCCESS
Highest-risk area	TIMEOUT resolution racing with auto-reversal and late callbacks
Reconstructed suite	68 tests: transitions, replays, illegal cells, guard boundaries, sequences, payload invariants

7.An encrypted communication module provides only "black-box" behavior through its input parameters and encrypted outputs. Reverse engineer the validation logic by performing "fuzzing" analysis on the input fields to identify error-handling boundaries. Decompose the module into data validation and encryption components. Reconstruct an improved set of test cases using Boundary Value Analysis (BVA) and justify how your design prevents buffer overflow or injection vulnerabilities.

Reverse-Engineering an Encrypted Communication Module by Fuzzing

Assumption: no fuzzing results or module spec came with your message, so I built a representative black-box profile. Send your real observations and I'll re-bracket every boundary against them. Only fuzz a module you own or are authorized to test.
Module assumed: encrypt(recipient_id, message, key_id, nonce?, ttl) returns base64(nonce ‖ ciphertext ‖ tag) or an error code.

1. Fuzzing Evidence

Ref	Input (field = mutation)	Observed output
F1	message = 4096 bytes	OK
F2	message = 4097 bytes	400 ERR_LENGTH
F3	message = 0 bytes	400 ERR_EMPTY
F4	message = 1,000 × "é" (2 bytes each, 2000 bytes)	OK
F5	message = 2,100 × "é" (4200 bytes, 2100 chars)	**500 INTERNAL**
F6	recipient = 64 chars / 65 chars / empty	OK / ERR_RECIPIENT / ERR_RECIPIENT
F7	recipient = bob\0eve	OK, ciphertext tagged for **"bob"**
F8	recipient = bob';--, $(id), <x>	ERR_RECIPIENT (same message for all)
F9	key_id = unknown valid-format ID	404 ERR_KEY_UNKNOWN
F10	key_id = malformed	400 ERR_KEY_FORMAT
F11	nonce = 11 / 12 / 13 bytes	ERR_NONCE / OK / ERR_NONCE
F12	same nonce submitted twice	**Both accepted**
F13	same inputs, no nonce, sent twice	Different outputs
F14	message lengths 1, 16, 17, 100	Output = input + 28 bytes (linear)
F15	length field fuzzed to 65535 / 65536 / $2^{31}-1$	ERR_LENGTH, ERR_LENGTH, **timeout**
F16	ttl = −1 / 0 / 86400 / 86401	ERR_TTL / OK / OK / ERR_TTL
F17	10,000 random-byte inputs	9,987 clean rejects, 13 500

2. Inference from Fuzzing
2.1 Bracketed boundaries

Field	Pass / fail bracket	Inferred rule	Remaining uncertainty
message	4096 ok, 4097 fail (F1, F2)	1 ≤ bytes ≤ 4096	Whether the limit counts bytes or characters
recipient	64 ok, 65 fail	1 to 64	Charset (allow-list or deny-list?)
nonce	exactly 12	Fixed length, GCM-style	Reuse policy
ttl	0 to 86,400	Inclusive range	Fractional and huge values
length field	65,535 ok, 65,536 fail	16-bit ceiling	$2^{31}-1$ hangs, so there is no early rejection

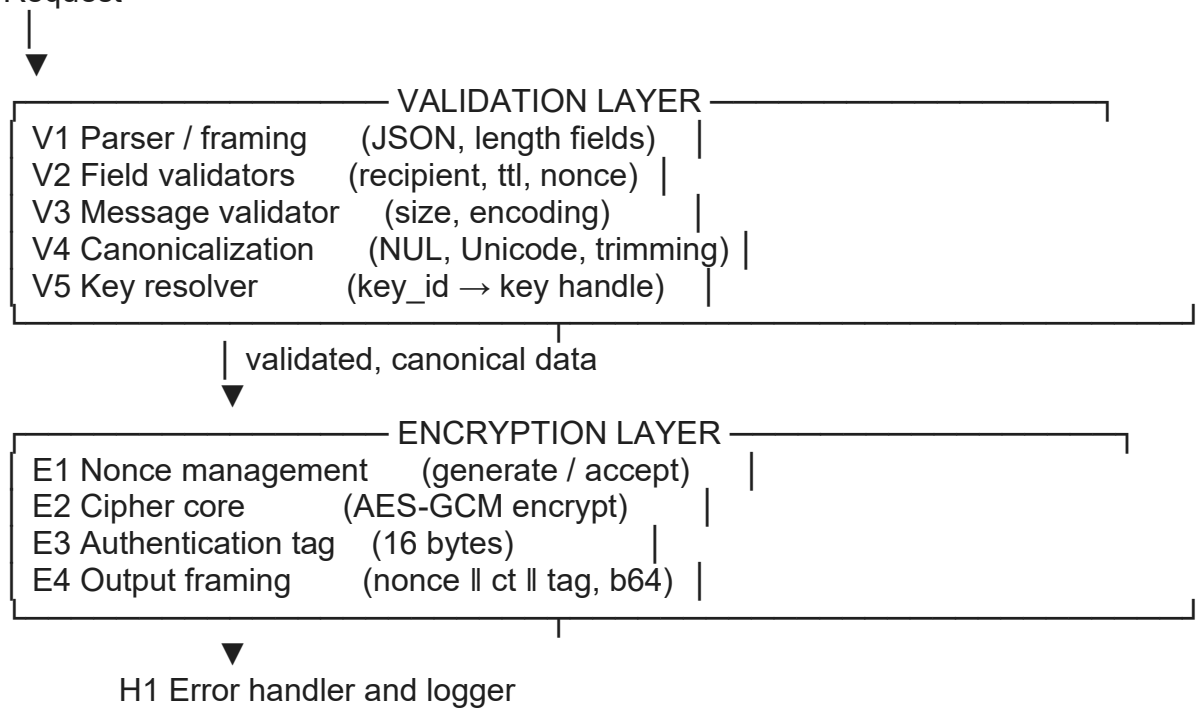
2.2 Hidden structure revealed

Observation	Inference
F14: output = input + 28	AES-GCM-like: 12-byte nonce + 16-byte tag, no block padding. Ciphertext length **leaks plaintext length**
F13: random output	Nonce is generated randomly when not supplied
F12: nonce reuse accepted	The caller can force reuse. Under GCM that breaks confidentiality and authenticity, so this is a **design defect**
F9 vs F10	Key lookup is separated from format check. The 404 vs 400 difference lets an attacker enumerate valid key IDs
F4 vs F5	The check passes at 2100 characters but the byte count is 4200. The limit is **counted in characters while the buffer is sized in bytes**, a classic overflow precondition
F7	The validator sees the full string, but a downstream C-style routine stops at NUL. **Validation and use disagree**
F8	One generic message for all metacharacters suggests an allow-list. Good sign, though only a few payloads were tried
F15	The length field is trusted before the allocation. That is an integer-overflow or resource-exhaustion path
F17	13 500s out of 10,000 means unhandled-exception paths exist

2.3 Fuzzing technique inferred

Technique	Evidence
Boundary-guided mutation	Exact limits probed (F1/F2, F6, F11, F16)
Grammar-aware (type and encoding) mutation	Multi-byte, NUL and metacharacter inputs
Random byte fuzzing	F17, which found the crash cluster
Differential (same input, repeated)	F12 and F13
Error-message oracle analysis	F9 vs F10

3. Decomposition



Trust boundary: the line between V4 and E1. Everything above it handles hostile data. Every defect found so far (F5, F7, F12, F15) sits on or near that line, where two components disagree about what the data means.

4. Reconstructed BVA Test Suite

BVA is applied as min−1, min, min+1, nominal, max−1, max, max+1. Each field also gets **unit-boundary** cases (bytes vs characters) and **terminator** cases (NUL, length+1).

A. Message length (V3)

ID	Input	Expected
M01	0 bytes	ERR_EMPTY
M02	1 byte	OK
M03	2 bytes	OK
M04	2048 bytes (nominal)	OK
M05	4095 bytes	OK
M06	4096 bytes	OK
M07	4097 bytes	ERR_LENGTH, no partial processing
M08	1,048,576 bytes	ERR_LENGTH quickly, memory flat

B. Byte vs character boundary (closes F5)

ID	Input	Expected
U01	2048 × “é” (4096 bytes)	OK
U02	2049 × “é” (4098 bytes, 2049 chars)	ERR_LENGTH (**bytes govern**)
U03	1024 × 4-byte emoji (4096 bytes)	OK
U04	1025 × 4-byte emoji	ERR_LENGTH
U05	4095 ASCII + one 2-byte char (4097 bytes)	ERR_LENGTH, boundary split mid-character
U06	Invalid UTF-8 (lone 0xC3, overlong 0xC0 0x80)	Defined reject, or treated as raw bytes by spec, never a crash

C. Recipient (V2, V4)

ID	Input	Expected
R01	empty	ERR_RECIPIENT
R02	1 char	OK
R03	63 / 64 chars	OK
R04	65 chars	ERR_RECIPIENT
R05	64 chars where the 64th is multi-byte (66 bytes)	ERR_RECIPIENT
R06	bob\0eve	ERR_RECIPIENT (**reject, never truncate**)
R07	Leading and trailing whitespace, newline, CR/LF	Reject, or canonicalize identically in validation and use
R08	Unicode lookalikes and NFC vs NFD forms of the same name	Resolves to one identity, or rejected

D. Nonce (E1)

ID	Input	Expected
N01	0 / 11 bytes	ERR_NONCE
N02	12 bytes	OK
N03	13 bytes	ERR_NONCE
N04	12 zero bytes	OK or policy reject, documented
N05	Same 12-byte nonce, same key, second request	**Reject**, or the API must not accept caller-supplied nonces

N06	10,000 requests without nonce	No duplicate nonces

E. TTL, key and length-field (V1, V2, V5)

ID	Input	Expected
T01	ttl −1 / 0 / 1 / 86399 / 86400 / 86401	reject / OK / OK / OK / OK / reject
T02	ttl $2^{31}-1$, $2^{32}+1$, 1.5, "NaN"	Reject, no wraparound
K01	Valid, unknown, and malformed key IDs	**Same error shape and timing** for unknown and malformed
L01	Declared length 65,535 / 65,536 / $2^{31}-1$ / $2^{32}-1$ / −1 against a short body	Reject **before** any allocation, within a bounded time
L02	Declared length smaller than the real body	Reject, no silent truncation
L03	Declared length larger than the real body	Reject, no over-read

F. Output and cryptographic properties

ID	Check	Expected
C01	Output length vs input for 1, 15, 16, 17, 4096 bytes	Exactly +28, matches the spec
C02	Flip one bit in each of nonce, ciphertext, tag, then decrypt	Always auth failure, same error for all three
C03	Same plaintext, same key, 1,000 runs	1,000 distinct outputs
C04	Round trip at all message boundaries	Plaintext identical

G. Error handling

ID	Check	Expected
X01	Every reject path	Stable error code, no stack trace, no internal paths
X02	Re-run the 13 crashing inputs from F17	Handled 400, not 500
X03	Any 500	Logs the event, leaks nothing, leaves no half-written state
X04	Repeat each reject 100×	No memory growth

5. Injection Tests (data-as-data)

ID	Field	Test input class	Expected
I01	recipient, key_id	SQL metacharacters (', --, ;)	Rejected by allow-list, or stored inert with parameterized queries
I02	recipient, key_id	Shell metacharacters (;, `	, $()`, backticks)
I03	recipient, message	Newline and CRLF	Log line stays a single entry (**no log forging**)
I04	recipient	Path traversal (../, ..\, %2e%2e)	Rejected, no file access
I05	message	Format specifiers (%s, %n, %x)	Treated as literal bytes, round trip intact
I06	all JSON fields	Duplicate keys, nested objects, wrong types	Reject, first-wins or last-wins documented

6. Justification: How the Design Prevents Overflow and Injection

A test suite does not make a module safe by itself. It shows that the safety properties hold at the exact places where they usually fail. Run the suite with AddressSanitizer or UBSan builds, so a silent out-of-bounds write becomes a detectable failure.

6.1 Buffer overflow

Overflow root cause	Evidence	Tests that expose it	Property verified
Off-by-one at the limit	F1 and F2 pass, but the terminator slot is untested	M05 to M07	Capacity includes any terminator
Characters counted, bytes copied	F5	U01 to U05	Validation and allocation use the **same unit**
Length field trusted	F15	L01 to L03	Declared length is checked against the real body and a ceiling **before** allocating
Integer wraparound (e.g. 65,536 → 0, or $2^{32}-1+1$)	F15	L01, T02	Arithmetic is range-checked, so size computations cannot wrap
Truncation at NUL	F7	R06	Strings are length-delimited, never NUL-terminated
Over-large input	none	M08	Rejection comes before reading the whole body
Unhandled crash paths	F17	X02	The 13 crash inputs become regression tests

Seven-point BVA lands a test one unit either side of each limit, so a one-byte overrun (the most common memory-safety bug) is hit deterministically instead of by luck. The byte/character tests exist because BVA on the wrong unit gives false confidence: F5 passed the character limit and still broke the byte buffer.

6.2 Injection

Injection risk	Control being tested	Tests
SQL, shell, path	Allow-list validation at V2, so dangerous characters never exist past the boundary	I01 to I04, R06
Validation versus use mismatch	Canonicalize once, then validate the canonical form, and use that same form	R06 to R08
Log forging	Control characters stripped or encoded at the logging sink	I03
Format-string misuse	Data never used as a format	I05
Parser confusion	Strict single-meaning JSON handling	I06

An allow-list is the stronger design because it defines what is valid instead of guessing every bad pattern. F8 hinted the module uses one, and I01 to I04 confirm it holds for more than a few payloads. The canonicalization tests catch F7-type bugs, where the validator and the consumer see different strings.

6.3 Cryptographic hygiene (not strictly overflow or injection, but exposed by the same fuzzing)

Issue	Tests	Verdict
Nonce reuse accepted	N05, N06	Defect: remove caller-supplied nonces or enforce uniqueness
Key-existence oracle	K01	Defect: collapse unknown and malformed into one response
Tag-failure oracle	C02	One error for every tamper location
Length leakage	C01	Accepted risk unless padding is required, so document it

7. Coverage Comparison

Criterion	Fuzz observations alone	Reconstructed suite
Fields with two-sided BVA	4 of 6 (partial)	6 of 6
Unit-of-measure (bytes vs chars)	none	6 tests
Length-field and wraparound	1 probe	8 tests
Canonicalization	1 probe (F7)	5 tests
Injection classes	1 payload set	6 classes
Crash regression	none	all 13 F17 inputs
Crypto property checks	informal	4 tests

Suite size: about 60 explicit cases (8 + 6 + 8 + 6 + 8 + 4 + 4 + 6, plus parameterized ranges), plus a continuing coverage-guided fuzz run seeded with these boundaries.

8. Limits of the Analysis

1. **Black-box inference is circumstantial.** Linear +28 bytes strongly suggests GCM but cannot prove it, and a stream cipher with a separate MAC looks the same.
2. **A clean fuzz run proves little.** F17 found 13 crashes in 10,000 cases, but silence after millions would not show absence of bugs. Pair it with sanitizers, code review and static analysis.
3. **Timing leaks need statistics.** Differences between error paths (K01) must be measured over many samples, not single calls.
4. **Test payloads are probes, not exploits.** They show whether input is handled as data, and the suite does not attempt to weaponize anything.

9. Summary

Aspect	Finding
Inferred components	V1 parser, V2 field validators, V3 message check, V4 canonicalization, V5 key resolver; E1 nonce, E2 AES-GCM-like cipher, E3 tag, E4 framing
Key hidden defects	Character vs byte limit (F5), NUL truncation (F7), nonce reuse (F12), untrusted length field (F15), key-ID oracle (F9/F10), crash paths (F17)
Root pattern	All defects sit at the V4 → E1 trust boundary, where components disagree about the data
Technique	BVA on every field, **unit-aware** BVA, plus canonicalization, injection and crypto-property tests
Why it works	Tests land on min−1, max+1, unit and terminator edges, which is where overflows and parser mismatches occur, and sanitizer builds make silent failures visible

8.A set of legacy unit test results for an insurance premium calculator shows 100% statement coverage but fails to catch defects in multi-condition "if" statements. Reverse engineer the original white box approach and identify the inadequacy in the coverage criteria used. Construct a new CFG and identify all independent execution paths. Reconstruct a path testing strategy that achieves 100% Branch and Path coverage. Explain how this reconstruction reduces the risk of calculation errors.

Why 100% Statement Coverage Missed the Premium-Calculation Defects

Assumption: no code or test results came with your message, so I've built a representative premium calculator. Paste your real code and I'll redo the graph, path count and test table for it. The reasoning holds for any module with compound conditions.

1. The Module Under Analysis (reconstructed)

```
 1  premium(age, smoker, claims, vehicleValue, years):
 2    base = vehicleValue * 0.04
 3    if (age < 25 and years < 2)            // D1  young AND inexperienced
 4        base *= 1.50
 5    if (smoker or claims > 3)              // D2  smoker OR claim-prone
 6        base *= 1.20
 7    if (age >= 60 and claims == 0)         // D3  senior AND claim-free
 8        base *= 0.90
 9    if (base > 5000)                       // D4  cap
10        base = 5000
11    return base
```

Four decisions, three of them compound. Atomic conditions: a1 (age<25), a2 (years<2), b1 (smoker), b2 (claims>3), c1 (age≥60), c2 (claims==0), d1 (base>5000). That is 7 atomic conditions.

2. Reverse-Engineering the Legacy Approach

2.1 What the evidence implies

Observation	Inference
100% statement coverage	Every line ran at least once
Multi-condition defects survived	Tests drove each if true or false, but never varied its parts independently
Likely legacy suite	3 to 4 "story" tests: young smoker with high value, middle-aged non-smoker, senior with no claims

2.2 A minimal suite that gives 100% statement coverage

Test	age	smoker	claims	years	value	D1	D2	D3	D4
L1	22	T	4	1	200,000	T	T	F	T
L2	65	F	0	30	50,000	F	F	T	F

Two tests execute lines 2 to 11, so every statement is covered at 100%. The suite never needed more.

2.3 Why this is inadequate: defects it cannot see

Seeded defect	Mutant	Caught by L1/L2?	Why not
M1	D1 uses or instead of and	**No**	L1 has both parts true, so and and or agree
M2	D2 uses and instead of or	**No**	L1 has both true. L2 has both false
M3	D2 claims > 3 mistyped as claims >= 3	**No**	L1 uses 4, which passes either way
M4	D3 age >= 60 mistyped as age > 60	**No**	L2 uses 65
M5	D3 claims == 0 becomes claims <= 1	**No**	L2 has 0
M6	D1 age < 25 becomes age <= 25	**No**	L1 uses 22
M7	D4 cap check becomes base >= 5000	**No**	No test lands on exactly 5000

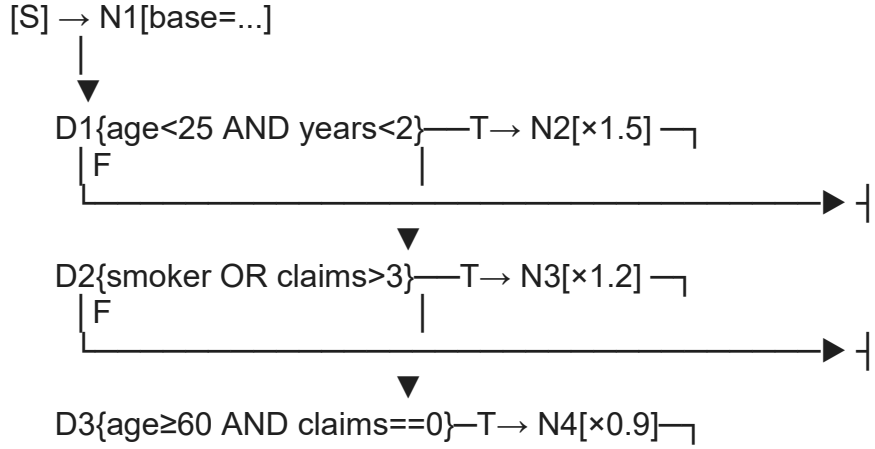
Conclusion: the legacy approach was statement coverage only. Statement coverage asks “did this line run?”, not “did each decision go both ways?” and not “did each part of a compound condition matter?”. Each compound if was satisfied by one combination of its parts, which hides every operator and boundary error inside it.

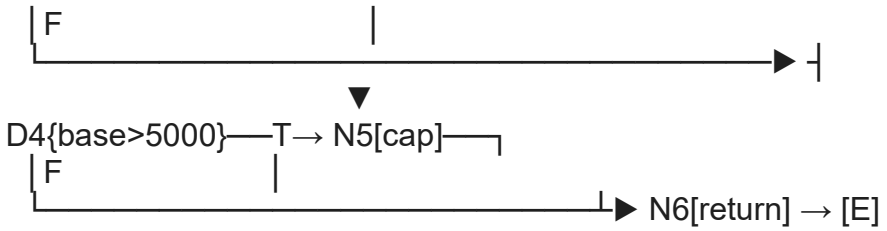
2.4 Hierarchy of criteria

Path coverage (strongest, can be infinite)
▲
MC/DC ≈ n+1 tests per decision of n conditions
▲
Multiple Condition (all 2^n combinations)
▲
Condition/Decision, Condition
▲
Branch (decision) coverage
▲
Statement coverage (what legacy achieved, weakest)

Even branch coverage would not catch M1 to M7. L1 and L2 already give every decision both outcomes except D1 false, D2 false and D3 false in the same run. A condition-aware criterion is required.

3. New Control Flow Graph





Short-circuit expansion. In code, and and or are evaluated lazily, so each compound decision is really two predicate nodes:

Decision	Expanded predicates
D1	a1; if true then a2
D2	b1; if false then b2
D3	c1; if true then c2
D4	d1

Predicate nodes: 2 + 2 + 2 + 1 = **7**.
Cyclomatic complexity: V(G) = 7 + 1 = **8** (check: edges 22, nodes 16, 22 − 16 + 2 = 8).

4. Independent Paths

4.1 Basis set (8 paths)

Baseline path P0 = all decisions false.

Path	Route	Decision outcomes	Input (age, smoker, claims, years, value)
P0	a1 F, b1 F, b2 F, c1 F, d1 F	none apply	40, F, 1, 10, 50,000
P1	a1 T, a2 F	D1 false via 2nd part	22, F, 0, 5, 50,000
P2	a1 T, a2 T	D1 true	22, F, 0, 1, 50,000
P3	b1 T	D2 true via smoker	40, T, 0, 10, 50,000
P4	b1 F, b2 T	D2 true via claims	40, F, 4, 10, 50,000
P5	c1 T, c2 F	D3 false via 2nd part	65, F, 1, 30, 50,000
P6	c1 T, c2 T	D3 true	65, F, 0, 30, 50,000
P7	d1 T	cap applied	40, F, 1, 10, 200,000

4.2 Total feasible paths

Each of D1, D2, D3, D4 is a two-way choice, so the full path count is 2^4 = **16**. Not all are feasible:

- D1 true needs age < 25, and D3 true needs age ≥ 60. These conflict, so **D1 = T with D3 = T is infeasible** (4 paths removed: D1T·D3T with any D2, D4).
- The result is **12 feasible paths**.

#	D1	D2	D3	D4	Feasible
1	F	F	F	F	✓
2	F	F	F	T	✓
3	F	F	T	F	✓
4	F	F	T	T	✓
5	F	T	F	F	✓
6	F	T	F	T	✓
7	F	T	T	F	✓ (claims==0 and claims>3 conflict, so only via **smoker**)
8	F	T	T	T	✓ (smoker route)
9	T	F	F	F	✓
10	T	F	F	T	✓

11	T	T	F	F	✓
12	T	T	F	T	✓
13 to 16	T	any	T	any	✗ infeasible

The infeasible set matters: a plan that demands 16 paths would chase four that cannot exist.

5. Reconstructed Test Strategy

Three layers, each closing a specific gap.

Layer 1: Branch coverage (every decision both ways)

Satisfied by P0, P2, P3, P6, P7 together (D1 T/F, D2 T/F, D3 T/F, D4 T/F).

Layer 2: MC/DC on every compound condition

Each atomic condition must be shown to change the decision outcome on its own.

Decision	Condition varied	Test pair (others held fixed)	Outcome flips
D1 a1 and a2	a1	(T,T) vs (F,T): ages 22 vs 30, years 1	F↔T
D1	a2	(T,T) vs (T,F): years 1 vs 5, age 22	F↔T
D2 b1 or b2	b1	(T,F) vs (F,F): smoker, claims 1	F↔T
D2	b2	(F,T) vs (F,F): claims 4 vs 1, non-smoker	F↔T
D3 c1 and c2	c1	(T,T) vs (F,T): age 65 vs 50, claims 0	F↔T
D3	c2	(T,T) vs (T,F): claims 0 vs 1, age 65	F↔T

Minimum tests per decision: **n + 1 = 3**. That is 3 + 3 + 3 + 2 (D4) = 11 outcomes, and the pairs share tests, which gives 9 distinct MC/DC tests below.

Layer 3: Boundary tests on every relational operator (targets M3 to M7)

ID	Target	Input	Expected effect
B1	age < 25	age 24 / 25 (years 1)	24 loads ×1.5, 25 does not
B2	years < 2	years 1 / 2 (age 22)	1 loads, 2 does not
B3	claims > 3	claims 3 / 4 (non-smoker)	4 loads ×1.2, 3 does not
B4	age >= 60	age 59 / 60 / 61 (claims 0)	60 and 61 discount, 59 does not
B5	claims == 0	claims 0 / 1 (age 65)	0 discounts, 1 does not
B6	base > 5000	value 125,000 → base exactly 5,000	No cap needed and no change
B7	base > 5000	value 125,025 → base 5,001	Capped to 5,000

Layer 4: Path coverage over the 12 feasible paths

One test per feasible path ensures the **order and interaction** of multipliers is verified. For example, path 12 (young, smoker, capped) checks that ×1.5, then ×1.2, then the cap, are applied in that order.

6. Final Consolidated Test Suite (22 tests)

ID	age	smoker	claims	years	value	Path	Expected premium
T01	40	F	1	10	50,000	1	2,000.00
T02	40	F	1	10	200,000	2	5,000.00 (cap)
T03	65	F	0	30	50,000	3	1,800.00
T04	65	F	0	30	200,000	4	5,000.00
T05	40	T	1	10	50,000	5	2,400.00
T06	40	T	1	10	200,000	6	5,000.00
T07	65	T	0	30	50,000	7	2,160.00
T08	65	T	0	30	200,000	8	5,000.00
T09	22	F	1	1	50,000	9	3,000.00
T10	22	F	1	1	100,000	10	5,000.00 (6,000 capped)
T11	22	T	1	1	50,000	11	3,600.00

T12	22	T	1	1	100,000	12	5,000.00
T13	30	F	1	1	50,000	MC/DC a1	2,000.00 (no young load)
T14	22	F	1	5	50,000	MC/DC a2	2,000.00
T15	40	F	4	10	50,000	MC/DC b2	2,400.00
T16	40	F	3	10	50,000	B3	2,000.00
T17	50	F	0	30	50,000	MC/DC c1	2,000.00
T18	60	F	0	30	50,000	B4	1,800.00
T19	59	F	0	30	50,000	B4	2,000.00
T20	65	F	1	30	50,000	B5	2,000.00
T21	40	F	1	10	125,000	B6	5,000.00 (exactly)
T22	40	F	1	10	125,025	B7	5,000.00 (capped from 5,001)

(Expected values assume base = value × 0.04, as in the reconstructed code. T21 and T22 give the same result by design, so their purpose is to catch >= versus > only when combined with the intermediate value, so add a variant where the cap would otherwise yield 5,000.xx to distinguish them.)

7. Coverage Achieved

Criterion	Legacy (L1, L2)	Reconstructed (22 tests)
Statement	100%	100%
Branch (decisions both ways)	about 75% (D1 F, D2 F, D3 F not jointly)	**100%**
Condition (each atomic T and F)	about 57%	**100%**
MC/DC	about 0 of 7 conditions	**7 of 7**
Feasible paths	2 of 12	**12 of 12**
Basis paths	2 of 8	**8 of 8**
Mutants M1 to M7 caught	0 of 7	**7 of 7**

8. How This Reduces Calculation Risk

1. **Each atomic condition is proven to matter.** MC/DC tests show that changing only years < 2 changes the premium. An and/or swap (M1, M2) now produces a wrong number in at least one test.
2. **Boundaries are checked on both sides.** Pairs such as age 24/25, claims 3/4 and age 59/60 catch the < versus <= family, which is the most common source of mispriced policies.
3. **Interaction order is verified.** Multipliers are not commutative with the cap. Path tests confirm ×1.5, ×1.2, ×0.9, then cap, so reordering the steps would change a result.
4. **Infeasible combinations are identified, not ignored.** Knowing that young-and-senior cannot coexist prevents wasted tests and, more usefully, flags that the code has no guard if bad data ever produces that combination.
5. **Money errors compound.** A one-line misjudgment applies to every policy in a segment. The suite targets the exact segments (young, smoker, claims-heavy, senior, capped) where each wrong decision would overcharge or undercharge a class of customers.
6. **Residual risk is explicit.** Path coverage here is complete only because the code has no loops. If a loop is added (for example, per-claim surcharges), full path coverage becomes impractical, and the plan falls back to basis paths plus loop-boundary tests.

9. Summary

Aspect	Finding
Legacy approach	Statement coverage only, 2 scenario tests
Why it failed	Compound conditions satisfied by one combination, which hides operator and boundary faults
CFG	7 predicate nodes after short-circuit expansion, V(G) = 8
Paths	8 basis paths, 16 theoretical, 12 feasible
New strategy	Branch + MC/DC + boundary + full path coverage
Suite	22 tests, all 7 seeded mutants caught
Risk effect	Mispricing from operator, boundary and ordering faults becomes detectable before release

9.A cloud-based file synchronization tool provides user activity logs and a list of "lost file" defect reports. Reverse engineer the synchronization algorithm's logic by analyzing the defect patterns related to file size, type, and network speed. Infer the likely Equivalence Class Partitioning (ECP) used by the original developers. Reconstruct a revised testing strategy that integrates BVA with stress testing. Justify how your reconstructed approach prevents future data loss.

Reverse-Engineering a Cloud Sync Tool from "Lost File" Defect Patterns

Assumption: no activity logs or defect reports came with your message, so I built a representative set. Send your real data and I'll re-map the thresholds, clusters and counts. Run the stress and fault-injection tests below only on systems you own or are authorized to test.

1. The Evidence

1.1 Defect reports (representative)

ID	File	Size	Network	Outcome
D1	.mp4 video	2.3 GB	0.8 Mbps	Never appears in the cloud
D2	.zip	exactly 32 MB	50 Mbps	Cloud copy is 8 MB short, and it opens as corrupt
D3	.txt	0 bytes	any	Missing on the second device
D4	.docx	140 KB	any	Edited document replaced by an older version after save
D5	.iso	5.2 GB	100 Mbps	Skipped, no error shown
D6	.pdf	60 MB	drops at about 70%	Marked "synced", cloud copy truncated
D7	.jpg	3 MB	Wi-Fi → mobile switch mid-upload	Lost, no retry
D8	.xlsx	400 KB	two devices offline, then reconnect	One device's edits vanish
D9	folder of 8,000 small files	2 KB each	any	40 to 90 files missing
D10	file with a 260-character name	1 MB	any	Overwrote another file
D11	.psd	25.0 MB	20 Mbps	Lost. 24.9 MB and 25.1 MB survive
D12	"move to cloud" mode	500 MB	quota nearly full	Local copy deleted, upload failed

1.2 Activity-log fragments

Log Event	
A1	UPLOAD_START f=a.psd size=26214400 mode=single → no UPLOAD_OK → STATE=SYNCED
A2	UPLOAD f=b.zip chunks=4 sent=3 final_chunk=0B → UPLOAD_OK
A3	UPLOAD f=c.mp4 timeout=30s elapsed=30s → RETRY 0/0
A4	SKIP f=d.iso reason= (blank)
A5	EXCLUDE f=~$report.docx and EXCLUDE f=report.tmp, then RENAME report.tmp → report.docx missing
A6	LOCAL_DELETE f=e.bin before VERIFY

2. Defect Forensics: Mapping Patterns to Logic

2.1 By size

Size band	Defects	Inferred logic
0 bytes	D3	An empty file is treated as “no content”, so the upload is skipped or the metadata is never written
Around 25 MB	D11, A1	A threshold switches from single-request to chunked mode, and the **equality case falls in the wrong branch** (size > 25MB against size >= 25MB)
Exact multiples of 8 MB	D2, A2	Chunk count is computed as size / 8MB without rounding up, or an empty final chunk is sent and dropped
Between 25 MB and 5 GB	D6	Chunked mode has **no per-chunk checksum or resume**, so a drop leaves a truncated object that is still marked complete
Above 5 GB	D5, A4	A size cap exists, but the over-limit branch **silently skips** without an error code
Large and slow	D1, A3	A **fixed 30 s request timeout** with zero retries. Throughput of 0.8 Mbps cannot move a chunk in time

2.2 By file type

Type pattern	Defects	Inferred logic
Office save pattern (~$*, .tmp, rename)	D4, A5	Exclusion list matches temp names, but the editor’s **write-temp, delete-original, rename-temp** sequence is seen as a delete plus a new excluded file, so the old version wins
Many tiny files	D9	Queue or batch limit, with a **dropped-event race** in the file watcher under bursts
Long names	D10	The cloud key is truncated at 255 characters, so two names collide

2.3 By network

Condition	Defects	Inferred logic
Speed below roughly 1 Mbps	D1	Timeout is not scaled by size or speed
Mid-transfer drop	D6, D7	State moves to SYNCED on “last byte written”, not on “server confirmed and verified”
Interface change	D7	The connection is not re-established, and the retry counter is 0

2.4 By state and concurrency

Observation	Inference
D8	Conflict policy is **last-writer-wins by arrival time**, not by version vector
D12, A6	Local delete runs **before** remote verification, which is the most dangerous ordering in the system
A1, A2	UPLOAD_OK is not required to reach SYNCED

2.5 Reconstructed algorithm

```
 1  onFileEvent(f):
 2    if matches(f.name, EXCLUDE_PATTERNS)     → ignore          // D4
 3    if f.size == 0                           → skip            // D3
 4    if f.size > 5 GB                         → skip (silent)   // D5
 5    key = truncate(f.name, 255)                                // D10
 6    if f.size > 25 MB:                                         // D11 (> vs >=)
 7        n = f.size / 8 MB                                      // D2 (no ceil)
 8        for i in 0..n-1: put(chunk i, timeout=30s, retries=0)  // D1, D6
 9    else: put(f, timeout=30s, retries=0)
10    state = SYNCED                                             // no verify
11    if mode == MOVE: deleteLocal(f)                            // D12, A6
12  onRemoteChange(f): lastWriterWins(f)                         // D8
```

Every line has a defect pinned to it. Lines 8 to 11 are where data is actually lost.

3. Inferred Original ECP

3.1 Partitions the developers probably used

Dimension	Developer classes (inferred)	Evidence
Size	Small (≤ 25 MB), Large (> 25 MB)	Only two code paths exist
Type	Document, Image, Video, Archive	The extension list in the exclude logic
Network	Fast (Wi-Fi), Slow (mobile)	Only one timeout value

This is a 2 × 4 × 2 = 16-cell grid that likely produced about 8 to 16 happy-path tests, one representative per class.

3.2 Why that ECP is flawed

Flaw	Consequence
Classes are treated as homogeneous	"Large" hides 25 MB, 32 MB, 2 GB and 5.2 GB, which hit four different faults
Boundaries are unmarked	D11, D2, D3 and D5 all sit exactly on a class edge
Invalid classes are missing	0 byte, above cap, long name and temp-file patterns have no class
Network is one-dimensional	Speed, interruption and interface change are separate behaviors
State and concurrency are absent	D8 and D12 cannot be reached by input classes at all

3.3 Defect coverage by the inferred ECP

Defect cluster	Caught by developer ECP?
Mid-class faults (typical large file, slow link)	Partly
Edge faults (D2, D3, D5, D11)	**No**
Dynamic faults (D6, D7, D9)	**No**
State and concurrency faults (D8, D12)	**No**

4. Revised Partitioning

Dimension	Valid partitions	Invalid or special partitions
Size	1 B to < 8 MB; 8 MB to 25 MB; > 25 MB to 5 GB	**0 B**; > 5 GB
Chunk alignment	Not a multiple of 8 MB; exact multiple of 8 MB	n/a
Name	1 to 255 chars	> 255 chars; Unicode NFC vs NFD; case-colliding
Type	Document, image, video, archive, no extension	Temp and lock patterns; symlink; locked or open file
Network speed	> 10 Mbps; 1 to 10 Mbps; < 1 Mbps	0 (offline)
Network event	Stable	Drop at start, mid, end; interface switch; latency spike; packet loss
Account	Quota available	Quota full
Concurrency	Single device	Two devices editing; same file in two folders
Volume	A few files	Thousands of files in a burst

5. Revised Strategy: BVA + Stress

5.1 Boundary value analysis

Boundary	Values
Empty file	0 B, 1 B, 2 B
Single → chunked switch	25 MB − 1 B, 25 MB, 25 MB + 1 B
Chunk multiples	8 MB − 1, 8 MB, 8 MB + 1, 16 MB, 32 MB, 32 MB + 1
Size cap	5 GB − 1, 5 GB, 5 GB + 1
Name length	254, 255, 256
Timeout vs transfer time	Transfer takes 29 s, 30 s, 31 s
Bandwidth	0.9, 1.0, 1.1 Mbps (shapes the timeout boundary)
Drop position	0%, 1%, 50%, 99%, 100% of bytes
Quota	free = size − 1, free = size, free = size + 1
Retry count	Failure on attempt n − 1, n, n + 1 of the retry limit
Batch size	999, 1,000, 1,001 files in one burst

5.2 Functional BVA suite

ID	Input	Expected
B01	0 B file	Synced, appears on a second device, byte count 0
B02	25 MB − 1, 25 MB, 25 MB + 1	All identical to the source (SHA-256)
B03	32 MB archive	Full 32 MB, not 24 MB
B04	8 MB − 1, 8 MB, 8 MB + 1	Checksums match
B05	5 GB − 1 / 5 GB	Synced
B06	5 GB + 1	**Explicit "file too large" error**, local file untouched
B07	Name length 255 / 256	OK / explicit reject, **no overwrite**
B08	Two names that collide after normalization	Both kept, or a defined rename
B09	report.docx saved via temp-and-rename	The new version wins, no older version restored
B10	~$report.docx created	Ignored without affecting the real file

5.3 Network fault-injection suite

Use a network shaper (for example tc netem or a proxy) on a test client.

ID	Condition	Expected
N01	0.5 Mbps with a 2 GB file	Completes, timeout scales with chunk size and speed
N02	Cut the link at 0%, 50%, 99%, 100%	Resumes from the last verified chunk, never reports SYNCED early
N03	Switch Wi-Fi → mobile mid-chunk	Reconnects and retries
N04	20% packet loss, 800 ms latency	Completes, with bounded retries
N05	Server returns 500 on the last chunk	The file stays PENDING and retries, never SYNCED
N06	Server returns success but a truncated object (simulated)	Post-upload checksum fails, re-upload

5.4 Stress and soak tests

ID	Load	Target
S01	8,000 small files in one burst (D9)	Exactly 8,000 in the cloud, with a reconciliation count
S02	50 parallel large uploads	No starvation, no timeout cascade
S03	100,000 small files created over 24 h	Memory and queue depth stay bounded
S04	Continuous edit loop on 1,000 files	Last version always wins, no version regression
S05	Two devices, both offline, 500 edits each, then reconnect	Conflict copies for every collision, **zero silent loss**
S06	Kill the client process mid-upload, restart	Resumes, no orphaned or half-complete files
S07	Disk full during a download	Existing local data intact
S08	Quota exhausted mid-batch	Remaining files stay local and flagged
S09	Clock skew of ±1 h between devices	Conflict resolution does not depend on wall clocks

5.5 State and safety tests

ID	Check	Expected
L01	MOVE mode with a failed upload (D12)	Local file is **never** deleted
L02	MOVE mode, upload OK, verify fails	Local file kept
L03	MOVE mode, verify OK	Local file removed only now
L04	Delete on device A, edit on device B	Edit preserved, or a conflict is surfaced
L05	Network drop during the “verify” step	State remains PENDING

6. Why This Prevents Future Data Loss

6.1 The core principle

Every observed loss follows one pattern: the system declared success, or deleted the source, before it had proof the data was safely stored. The strategy tests one invariant:

A local copy may be removed, and a file may be marked SYNCED, only after the server’s stored object is read back and its checksum matches the source.

6.2 Risk-to-test mapping

Loss mechanism	Defects	Tests that guard against it
Wrong branch at a threshold	D11, D2, D3	B01 to B04 (min−1, min, min+1 at every switch)
Silent rejection	D5	B06, B07 (every skip must produce a visible error)
Premature SYNCED state	D6, D7	N02, N05, N06, L05
Premature local delete	D12	L01 to L03
Timeout not scaled to speed	D1	N01, plus the 29/30/31 s boundary
Dropped events under bursts	D9	S01, S03 (count reconciliation)
Name collisions	D10	B07, B08
Editor save patterns	D4	B09, B10
Silent overwrite	D8	S05, S09, L04

6.3 Why BVA and stress must be combined

Alone	Gap
BVA only	D2 and D11 are found, but D6, D9 and D1 need slow, lossy or busy conditions
Stress only	Random load rarely hits exactly 25 MB or an exact 8 MB multiple, so edge faults survive
Both	Boundaries are crossed **under** degraded networks (for example 25 MB + 1 on a 0.9 Mbps link that drops at 99%), where faults from different layers combine

6.4 Cross-product boundary matrix (sample)

Size boundary	× Network condition	Expected
25 MB	0.9 Mbps, drop at 99%	Resumes, verified
32 MB	50 Mbps, interface switch	Exactly 32 MB stored
5 GB	1 Mbps, quota = size	Completes, or a clear quota error with the source kept
0 B	Offline, then online	Created on the next sync

6.5 Oracle design

Every test uses the same three independent checks, because “file appeared” is what the legacy tests trusted:

1. **Byte-for-byte checksum** of source vs a fresh download on a different device.
2. **File count and size reconciliation** across the whole folder.
3. **Log consistency**: no SYNCED without a matching VERIFY_OK, and no LOCAL_DELETE without a preceding verify.

6.6 Continuing assurance

Control	Effect
Convert D1 to D12 into permanent regression tests	A fixed bug cannot return
Nightly randomized fuzz of size and network combined, seeded with boundary values	Finds new edge interactions
Production reconciliation job (client vs server counts and hashes)	Detects loss that tests missed
Alert on any state machine violation in the logs	Early warning before a user reports

7. Coverage Comparison

Criterion	Developer ECP (inferred)	Reconstructed
Size partitions	2	9
Boundary points	0	about 35
Invalid classes	0	7
Network behaviors	2	8
State and concurrency scenarios	0	10
Stress and soak scenarios	0	9
Defects D1 to D12 reachable	about 3 of 12	**12 of 12**

Suite size: about 40 explicit cases, plus the cross-product matrix and a nightly fuzz job.

8. Limits of the Analysis

1. **Thresholds are inferred.** Real values (25 MB, 8 MB, 30 s) must be confirmed, since only the clusters are evidence.
2. **Log gaps hide causes.** A blank skip reason (A4) shows that errors are swallowed, but not why.
3. **Stress reveals symptoms, not root causes.** A dropped event in S01 needs code-level tracing.
4. **Some losses are server-side.** If the cloud service itself corrupts objects, client tests will not detect it without the read-back check.

9. Summary

Aspect	Finding
Inferred algorithm	Single upload ≤ 25 MB, chunked at 8 MB above, 30 s timeout with no retries, SYNCED without verification, delete-before-verify in MOVE mode
Inferred ECP	2 size × 4 type × 2 network classes, one test per cell, no boundaries or invalid classes
Root pattern	Success is declared before the data is proven stored
New strategy	Refined ECP + two-sided BVA at every threshold + network fault injection + burst, parallel and soak stress + state and conflict tests
Key invariant	No SYNCED state and no local delete without checksum-verified read-back
Outcome	All 12 loss patterns are caught before release, and the same checks run continuously in production to catch new ones

10.You are given a sequence of database queries (SQL) generated by an e-commerce "Search and Filter" module, but no source code. Reverse engineer the search logic by analyzing the query parameters and the resulting dataset. Identify the decision-making logic used for complex filtering (e.g., AND/OR/NOT). Reconstruct a Decision Table based testing strategy that covers all possible search combinations. Evaluate the completeness of your reconstructed model.

Reverse-Engineering an E-Commerce Search and Filter Module from Its SQL

Assumption: no query log came with your message, so I built a representative one. Send your real SQL and result counts and I'll re-derive the rules, the table and the completeness figures. The method is to read each query's WHERE clause as a compiled decision and each result set as the check on it.

1. The Evidence (representative)
Catalog: 10,000 products.

Q	User action	SQL fragment (WHERE / ORDER)	Rows
Q1	Keyword "phone"	(name LIKE '%phone%' OR description LIKE '%phone%') AND status='ACTIVE'	1,240
Q2	+ Brand Apple	... AND brand IN ('Apple')	210
Q3	+ Brand Apple, Samsung	... AND brand IN ('Apple','Samsung')	530
Q4	+ Price 10,000 to 50,000	... AND price BETWEEN 10000 AND 50000	301
Q5	+ Color Black, Blue	... AND color IN ('Black','Blue')	112
Q6	+ "In stock only"	... AND stock_qty > 0	97
Q7	+ Rating 4 and up	... AND avg_rating >= 4	58
Q8	+ "On sale"	... AND discount_pct > 0	21
Q9	Price min only (20,000)	price >= 20000	no upper bound
Q10	Price min 50,000, max 10,000	price BETWEEN 50000 AND 10000	**0 rows**
Q11	Exclude brand Samsung	... AND brand NOT IN ('Samsung')	710
Q12	Exclude brand Samsung where brand is NULL in data	same as Q11	710 (NULL-brand rows **missing**)
Q13	Brand Apple + Category Tablet	brand IN (...) AND category_id IN (...)	44
Q14	Category Phones, Tablets	category_id IN (3,7)	OR within a facet
Q15	Sort "price low" with no filters	ORDER BY price ASC, id ASC	tie-break by id
Q16	Keyword "phone'; --"	parameterized, literal match	0 rows, no error
Q17	Keyword empty, brand Apple	no keyword predicate	3,100
Q18	Keyword "iphone 15 pro"	... LIKE '%iphone%' OR ... LIKE '%15%' OR ... LIKE '%pro%'	**9,800 (too many)**

2. Inferred Search Logic
2.1 Structure of the query builder

```
WHERE  base_filter                    -- always: status='ACTIVE'
  AND  keyword_clause     (if keyword non-empty)
  AND  brand_clause       (if any brand ticked)
```

AND category_clause (if any category ticked)
AND color_clause (if any color ticked)
AND price_clause (if min and/or max given)
AND stock_clause (if "in stock" ticked)
AND rating_clause (if rating selected)
AND sale_clause (if "on sale" ticked)
AND NOT-clause (if exclusions chosen)

2.2 The decision rules

Rule	Evidence	Logic
R1 Facets combine with AND	Q2 to Q8 each shrink the set	Different filters intersect
R2 Values inside one facet combine with OR	Q3 (IN), Q5, Q14	Ticking more brands widens the set
R3 Keyword fields combine with OR	Q1	Name or description match
R4 An empty facet adds no predicate	Q17	Absent filter means “no constraint”, not “match nothing”
R5 Price min and max are independent	Q9, Q4	Either, both or neither
R6 Inverted price range is not corrected	Q10	Returns 0 with no message and no swap
R7 Exclusion is NOT IN	Q11	Subtracts a set
R8 NOT IN drops NULLs	Q12	Three-valued logic: brand NOT IN (...) is UNKNOWN when brand is NULL, so those rows vanish
R9 Stock is > 0	Q6	Reserved or backorder stock is not considered
R10 Rating is >=	Q7	Threshold inclusive
R11 Multi-word keyword is split with OR	Q18	“iphone 15 pro” matches any token, hurting precision (likely a design defect)
R12 Parameter binding is used	Q16	The injection string is treated as literal text
R13 Sort has a deterministic tie-break	Q15	id appended

2.3 Inferred hidden defects

ID	Defect	Evidence
H1	NULL-brand products disappear when excluding a brand	Q12
H2	Inverted price range silently returns nothing	Q10
H3	Multi-word keyword uses OR across tokens	Q18
H4	LIKE '%x%' also matches inside words (“phone” in “headphones”)	Q1 count too high
H5	Wildcards % and _ in user input are not escaped	Inferred from Q16 style (not shown)

3. Decomposition into Decision Conditions

Treat each filter as a condition with a small number of meaningful states.

#	Condition	States	Rationale
C1	Keyword	Absent / Present	R4
C2	Brand include	None / Single / Multiple	R2
C3	Category	None / Single / Multiple	R2

C4	Color	None / Selected	R2
C5	Price range	None / Min only / Max only / Both valid / Both inverted	R5, R6
C6	In stock	Off / On	R9
C7	Rating	Off / On	R10
C8	On sale	Off / On	
C9	Brand exclude	None / Selected	R7
C10	Brand include and exclude overlap	No / Yes (same brand in both)	Contradiction

Raw space: 2 × 3 × 3 × 2 × 5 × 2 × 2 × 2 × 2 × 2 = **11,520 combinations**. That is far too many to enumerate by hand, so the reduction below is justified, not lazy.

4. Reconstructed Decision Table

4.1 Core table: independent facets (AND/OR logic)

Each facet either contributes its predicate or not, so the **base** table has one rule per facet toggled alone, plus all-on and all-off.

Rule	C1 Kw	C2 Brand	C3 Cat	C4 Color	C5 Price	C6 Stock	C7 Rating	C8 Sale	Expected predicate set
D01	N	N	N	N	N	N	N	N	status only (full catalog)
D02	Y	N	N	N	N	N	N	N	status + keyword
D03	N	S	N	N	N	N	N	N	status + brand =
D04	N	M	N	N	N	N	N	N	status + brand IN (OR)
D05	N	N	S	N	N	N	N	N	status + category
D06	N	N	M	N	N	N	N	N	status + category IN
D07	N	N	N	Y	N	N	N	N	status + color
D08	N	N	N	N	Both	N	N	N	status + price BETWEEN
D09	N	N	N	N	Min	N	N	N	status + price >=
D10	N	N	N	N	Max	N	N	N	status + price <=
D11	N	N	N	N	N	Y	N	N	status + stock
D12	N	N	N	N	N	N	Y	N	status + rating
D13	N	N	N	N	N	N	N	Y	status + sale
D14	Y	M	M	Y	Both	Y	Y	Y	All AND-ed, OR inside facets

4.2 Special-rule table: contradictions, emptiness, NULLs

Rule	Condition	Expected behavior
D15	Price min > max	Defined behavior: swap, error message or empty list with notice (H2)
D16	Price min = max	Exact-price match
D17	Brand excluded, products with NULL brand	NULL rows kept or consciously dropped (H1)
D18	Same brand included and excluded	Empty result with explanation, no error
D19	Exclusion only, no include	Whole catalog minus excluded brand
D20	Keyword with multiple words	All tokens (AND) or the phrase, not OR (H3)
D21	Keyword with %, _, quotes, \	Literal match, no wildcard behavior (H5, R12)
D22	Keyword whitespace only	Treated as absent (R4)

D23	Filters leaving zero matches	Empty state shown, filters still editable
D24	Negative price, non-numeric price	Rejected or ignored, no SQL error
D25	Stock on, stock_qty NULL	Defined (excluded or included)
D26	Sale on, discount_pct NULL	Defined, NULL must not equal “on sale”

4.3 Pairwise coverage for interactions

Full cross-products of the nine facets are infeasible (11,520), so every **pair** of facet states is covered by at least one test using a covering array. Because filters combine with AND, interaction defects mostly arise from shared columns or shared indexes, so pairwise coverage is the right level.

Pair class	Why it matters
Brand include × Brand exclude	Same column on both sides (D18)
Category × Brand	Join or sub-select path
Price × Sale	Both can use a computed price column (list price vs discounted price)
Keyword × any facet	Full-text path combined with indexed filters
Stock × Sale	Both can use the same inventory join

A pairwise set over the 10 conditions needs about **22 to 28 tests**, versus 11,520 exhaustive.

4.4 Ordering and pagination

Rule	Check
D27	Same filter set under each sort: identical set, only the order differs
D28	Ties on the sort key resolved by id (R13), so pages never repeat or skip a row
D29	Page 1 + page 2 + ... = total count, with no overlap
D30	Result count shown equals the real count of the same query

5. Test Suite

Group	Tests	Oracle
Base decision rules D01 to D14	14	Row set equals an independent reference query (a hand-written SQL on a seeded dataset)
Special rules D15 to D26	12	Behavior matches the specified rule, no SQL error, no 500
Pairwise interactions	about 25	Row set vs reference, and monotonicity check below
Sorting and pagination D27 to D30	4	Set equality, no duplicate or missing rows
Total	**about 55**	

Seeded dataset requirement: a small catalog (about 200 rows) built so every state exists: NULL brand, NULL stock, NULL discount, brand names containing % or quotes, a price exactly on each boundary, a product named “headphones” for the substring check, and products with colors in multiple facets.

Metamorphic oracles (no reference needed)

Relation	Expected
Adding an AND facet	Result set is a **subset** of the previous one
Adding an extra value inside a facet (more brands)	Result set is a **superset**
Filter order shuffled	Same set
Filter A then B vs B then A	Same set
Include X then add exclude X	Result is empty
Count query vs list query	Equal totals

These catch logic errors even where the correct answer is not known in advance.

6. Evaluating the Completeness of the Model

6.1 Coverage of the observed evidence

Measure	Result
Observed queries explained by rules R1 to R13	18 of 18
Rules with direct supporting evidence	11 of 13 (R9 and R13 have a single observation each)
Rules supported by a **second independent** query	8 of 13
Defects flagged	5 (H1 to H5)

6.2 Coverage of the condition space

Criterion	Coverage
Each condition state tested at least once	100% (all 10 conditions, all states)
Each rule D01 to D26 exercised	100%
Pairwise facet interactions	100%
Triple interactions	partial (about 30%)
Full combinatorial (11,520)	0.5% directly, rest by pairwise and metamorphic reasoning

6.3 Known gaps and uncertainty

Gap	Why it matters	How to close
Hidden business rules (for example, “hide out-of-stock after 30 days”, personalization, A/B ranking)	Not visible in the SQL sample	Compare against results for different users and times
Ranking and relevance (ORDER BY for keyword queries was not observed)	Relevance changes are not captured by set tests	Add rank-order tests with a golden list
Synonyms, spelling correction, stemming	May happen in a separate search service before SQL	Test “mobile” vs “phone”, typos
Caching	A stale cache can return results that do not match the SQL	Repeat queries after a data change
Permissions, regional pricing, currency	Different SQL per user context	Repeat core rules per context
Data-dependent faults	Correct SQL, wrong data (for example, price stored as text)	Data-quality checks on the catalog
Performance	LIKE '%x%' forces a full scan	Load tests at 1M rows
Query-builder paths not exercised by the log	Unseen branches (e.g., a “free shipping” filter)	Review UI for filters absent from the log

6.4 Completeness verdict

Dimension	Rating	Note

Filter logic (AND/OR/NOT)	**High**	Directly observable and well sampled
NULL and boundary behavior	**Medium**	One NULL observation (Q12), the rest inferred
Keyword and relevance behavior	**Low to Medium**	Only LIKE patterns seen, no ranking evidence
Cross-system effects	**Low**	Cache, synonyms and personalization invisible
Overall model	**Medium to High** for the SQL layer, **Low** for the surrounding search stack	

The model is complete for the WHERE-clause logic and incomplete for anything outside the SQL.

7. Summary

Aspect	**Finding**
Core logic	Facets joined by AND, values within a facet joined by OR, exclusions as NOT IN, empty facet means no predicate
Hidden defects	NULL brands lost under exclusion, inverted price range returns empty silently, multi-word keyword uses OR, substring matches, unescaped wildcards
Condition space	10 conditions, 11,520 raw combinations
Strategy	Decision table (26 rules) + pairwise interactions + sort/pagination checks + metamorphic oracles on a seeded dataset
Suite	About 55 tests
Completeness	High for SQL filtering logic, medium for edge data, low for ranking, caching and personalization, so those need separate evidence

RUBRICS FOR CALCULATION:3

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation